Planning and Reinforcement Learning in AI Systems

(PARLAI)

Lesson 4a- Model-based Planning - Fully Observable Deterministic

Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

What is Model-based Planning

Model-Based Planning

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Model-based planning is a decision-making approach in which an agent uses a **model of the environment** to reason about future outcomes before acting.

The model typically includes a **dynamics model** that describes how the state evolves as a result of action execution, and a **reward \cost model** which associates a numeric signal to states and actions.

Using this model, the agent can:

- simulate possible future trajectories
- evaluate the consequences of candidate actions
- choose the action that leads to the best expected outcome

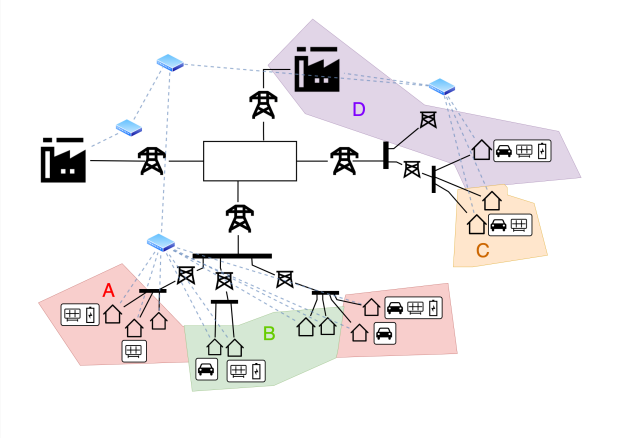
Model-based planning = **using a model to think ahead before acting**

Planning Examples

- Route selection
- Solving 15-puzzle, Rubik's cube, etc.
- Selecting and ordering movements of an elevator or a crane
- Production lines control
- Autonomous robots
- Crisis management
- ...

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Planning Examples



What is Model-based Planning

Planning Challenges

Approaches

Uninformed Search

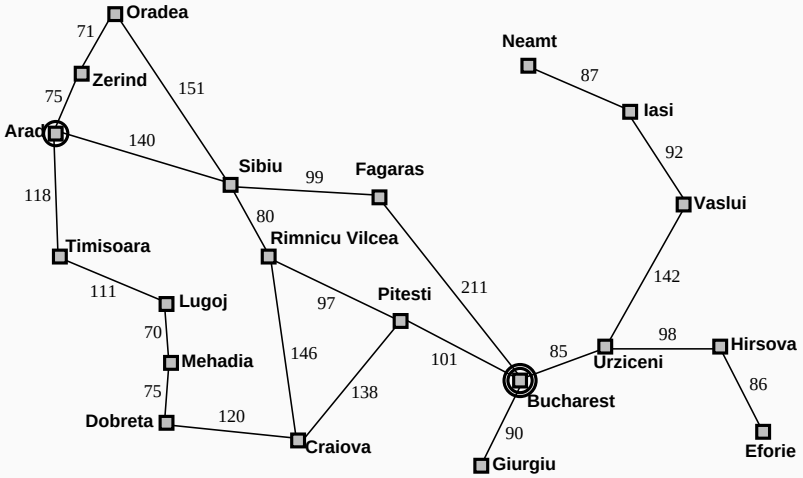
Informed Search

Local search

Evaluating Planning Algorithms

Example: A trip in Romania

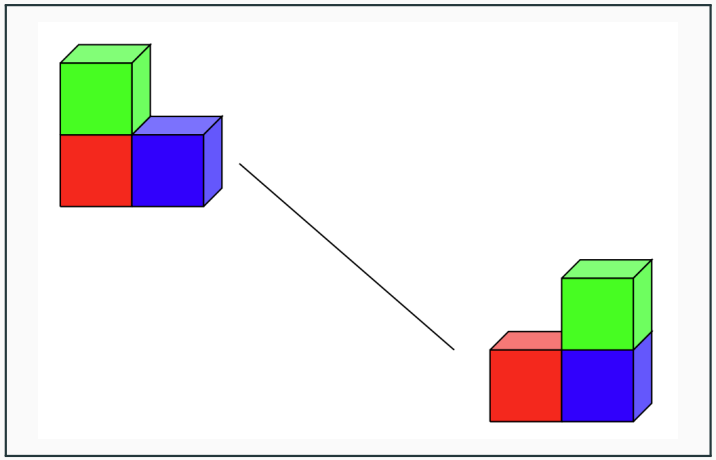
You are on holiday in Romania, currently in Arad. Your flight leaves tomorrow from Bucharest.



- **Goal**: be in Bucharest
- **States**: various cities
- **Actions**: drive between cities
- **solution**: sequence of cities, e.g., Arad, Sibiu, Fagaras, Bucharest

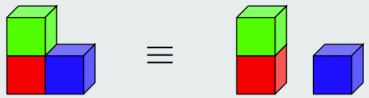
What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Example: Blocks world

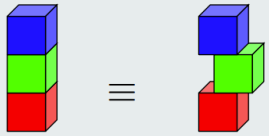


The rules of the game

Location on the table does not matter.

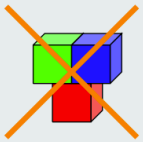


Location on a block does not matter.



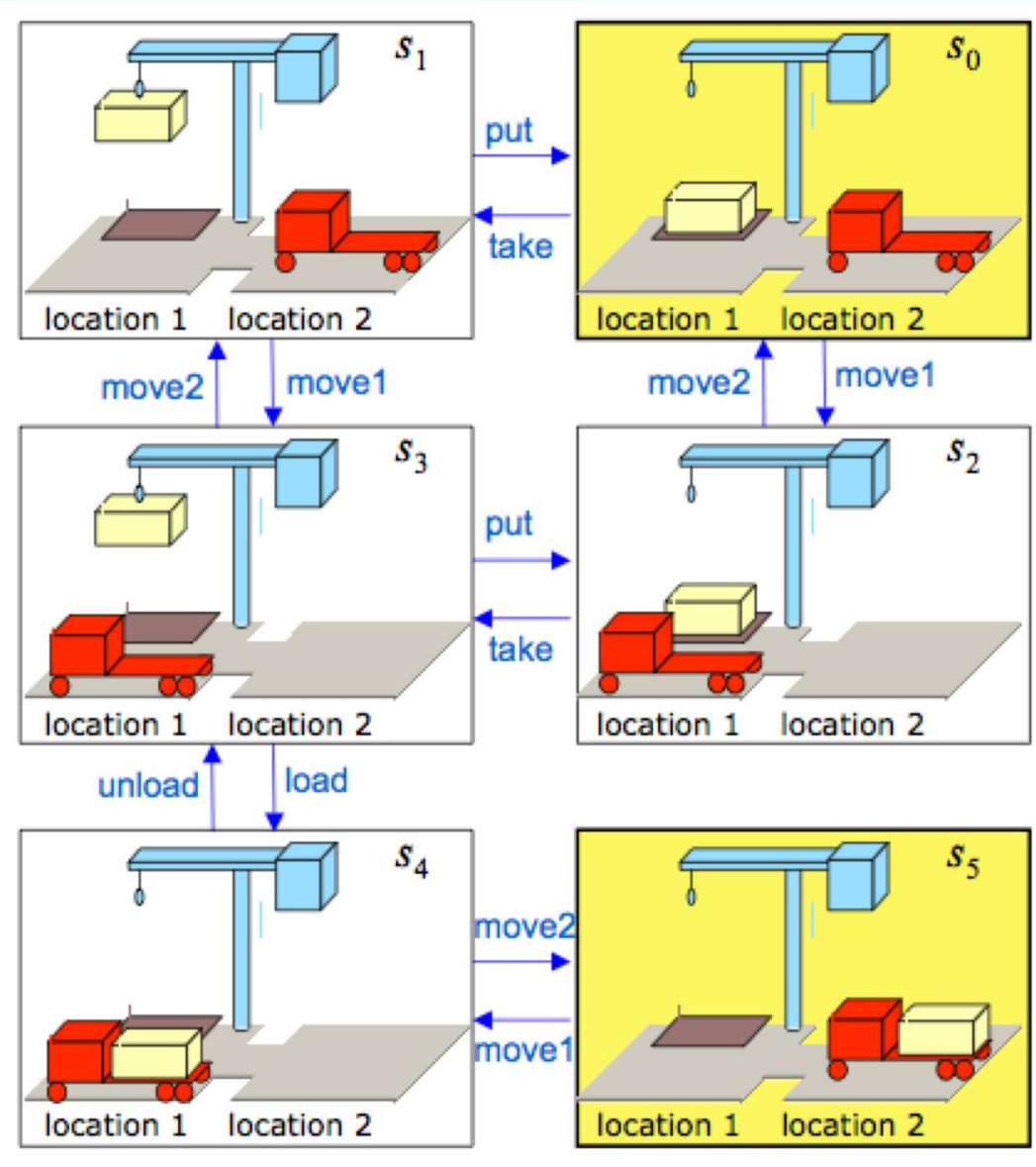
At most one block may be below a block.

At most one block may be on top of a block.



What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Example: Dock Logistics



What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Model-Based vs. Model-Free Methods

Model-based Planning

- Uses an explicit model.
- Plans via simulation \prediction.
- Sample efficient.
- Computational cost at decision time.

Model-free Decision-making

- Learn policy or value directly
- Limited lookahead reasoning

Modern methods often combine both:
learn a model + plan with it

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Model-Based vs. Model-Free: Intuition

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Example: Navigation in a city

- **Model-based agent:**
  - has access to a map (model of roads and distances)
  - simulates possible routes before moving
  - selects the shortest or fastest path
- **Model-free agent:**
  - has no map
  - learns from experience which actions work well
  - improves behavior via trial-and-error

Model-based = **plan using a model**
Model-free = **learn from experience**

Planning vs. Scheduling

Planning can be seen as extending scheduling.

Scheduling: Deciding **when** to perform a given set of actions while considering:

- Time constraints
- Resource constraints
- Global constraints (e.g., regulatory issues)
- Objective functions

Planning: Deciding **what** actions to perform (and **when**) to achieve a given objective.

Same issues as in scheduling. The difference is in solution techniques and worst-case time/space complexity.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Model-Based Planning

Characterized by:

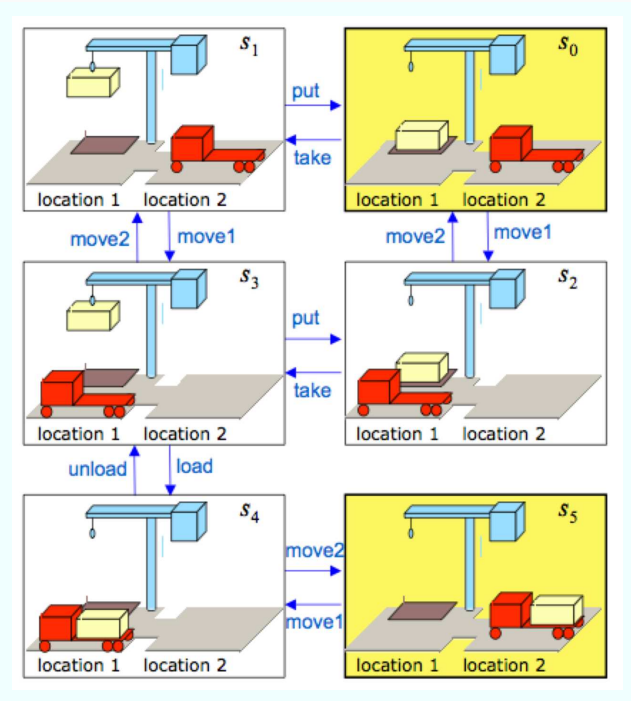
- A set of **states** $\mathcal{S}$ a system can be in.
  - a state is a full assignment to a set of **variables** (features) $\mathcal{X}$.
- **Actions** $\mathcal{A}$ change the values of certain variables.
- **Reward Function** $\mathcal{R}$ sets a numeric signal passed from the environment (can represent cost):
  - used to signal the objective
  - some domains have **goal conditions** such that the agent should reach **goal states** that satisfy them (e.g., 'be at Austin').
- **Objective:** find a **policy** that drives the initial state into a goal state / maximizes the expected accumulated reward.
- **Language**: generic rather then domain specific.
- **Complexity:** Even in the simplest setting, it is NP-hard, i.e., exponential in the number of variables in the worst case.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

State Model for Deterministic planning

- Finite state space $\mathcal{S}$ (typically defined via a feature space $\mathcal{X}$).
- An initial state $s_0 \in \mathcal{S}$
- A set $\mathcal{S}_G \subseteq \mathcal{S}$ of goal states
- Applicable actions $\mathcal{A}(s) \subseteq \mathcal{A}$ for $s \in \mathcal{S}$
- A transition function $s' = f(a, s)$ for $a \in \mathcal{A}(s)$
- A cost function $\mathcal{C} : \mathcal{A} \to [0, \infty)$

A **solution** is a sequence of applicable actions that maps s_0 into $\mathcal{S}_G$
An **optimal solution** minimizes the accumulated cost $\mathcal{C}$



Deterministic transition systems

A transition system is **deterministic** if there is only **one initial state** and all **actions are deterministic**. Hence, all future states of the world are assumed to be completely predictable.

Definition (deterministic transition system)

A **deterministic transition system** is $\langle S, s_0, A, S_G, c \rangle$ where

- finite state space S
- an initial state $s_0 \in S$
- a set $S_G \subseteq S$ of goal states
- applicable actions $A(s) \subseteq A$ for $s \in S$
- a transition function $s' = f(a, s)$ for $a \in A(s)$
- a cost function $c : A^* \to [0, \infty)$

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Reminder: Propositional STRIPS

Deterministic fully observable domains can be defined using **Propositional STRIPS** (Fikes and Nilsson 1971) where a task is a tuple $P = \langle \mathcal{F}, I, \mathcal{A}, G, \mathcal{C} \rangle$, where

- $\mathcal{F}$ is a set of features (features) and a state s is represented by the features that are true in s
- $I \subseteq \mathcal{F}$ is the initial state
- $G \subseteq \mathcal{F}$ represents the set of goal states, and
- $\mathcal{A}$ is a set of actions.
  - Each action is a triple $a = \langle pre(a), add(a), del(a) \rangle$, which represents the precondition, add, and delete lists respectively, all subsets of $\mathcal{F}$.
  - An action a is applicable in state s if $pre(a) \subseteq s$.
  - If action a is applied in state s, it results in a new state $s' = (s \setminus del(a)) \cup add(a)$.
- $\mathcal{C} : \mathcal{A} \to \mathbb{R}_0^+$ is a cost function that assigns each action a non-negative cost.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Planning Challenges

Why planning is difficult?

Limited model availability: In many settings, the underlying model is unknown, must be learned from data, is only a coarse approximation of reality, or is difficult to represent compactly.

Long-term dependencies: Actions might have delayed and long-term effects.

Uncertainty: Decisions might be based on partial and noisy observations, and actions may have stochastic outcomes.

Combinatorial explosion: The number of states and possible action sequences grows exponentially.

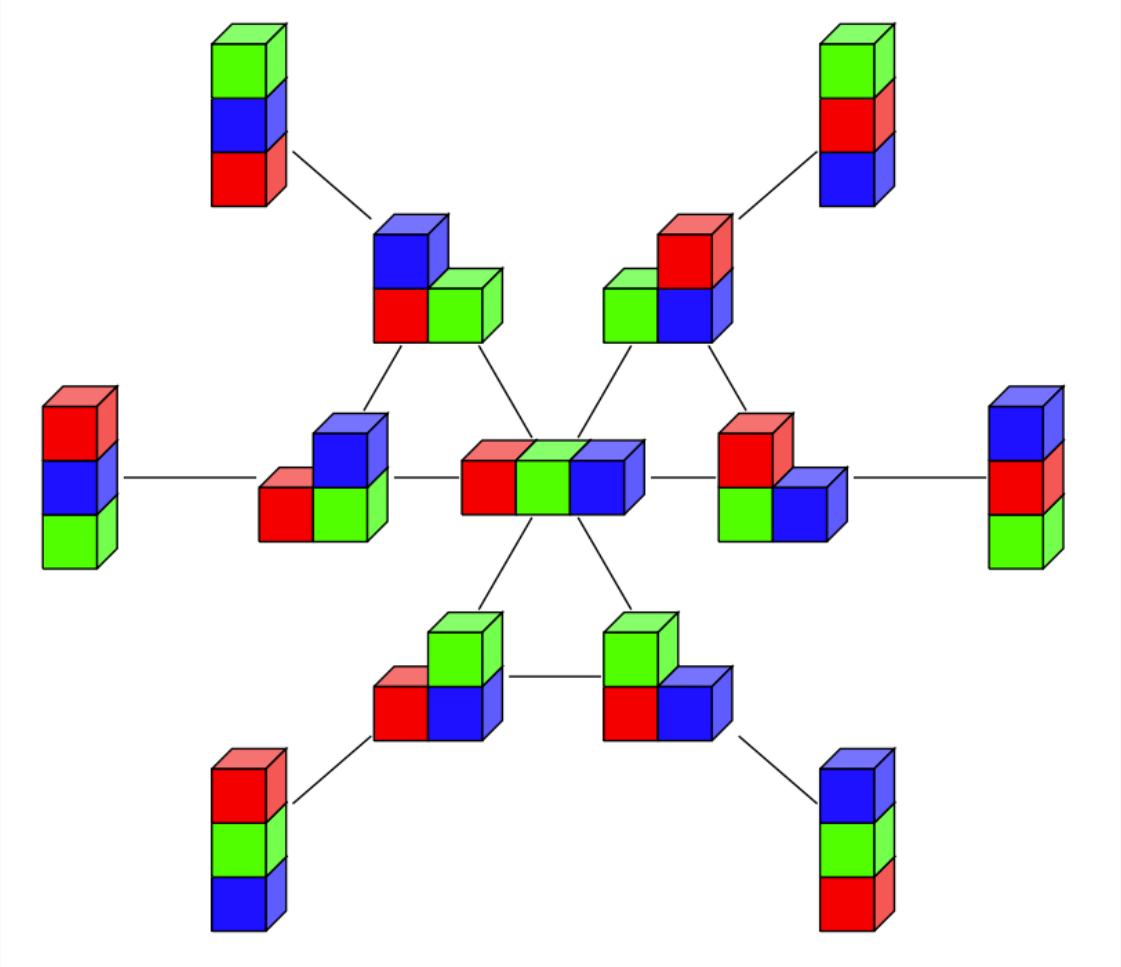
Computational constraints: Planning often needs to be performed under limited time and resources.

Planning requires balancing **expressiveness**, **uncertainty**, and **computational tractability**.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Why planning is difficult?

The transition graph for three blocks

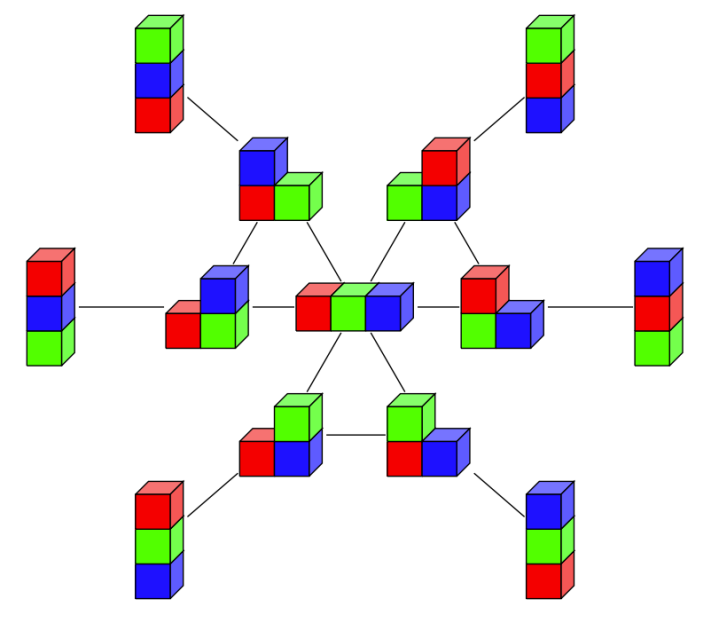


What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Why planning is difficult?

Solutions to deterministic planning problems are paths from an initial state to a goal state in the implicit **transition** graph, i.e., the graph that represents states and action-induced transitions between them.

Dijkstra's algorithm solves this problem in $O(|V| \log(|V|) + |E|)$



Are we done?

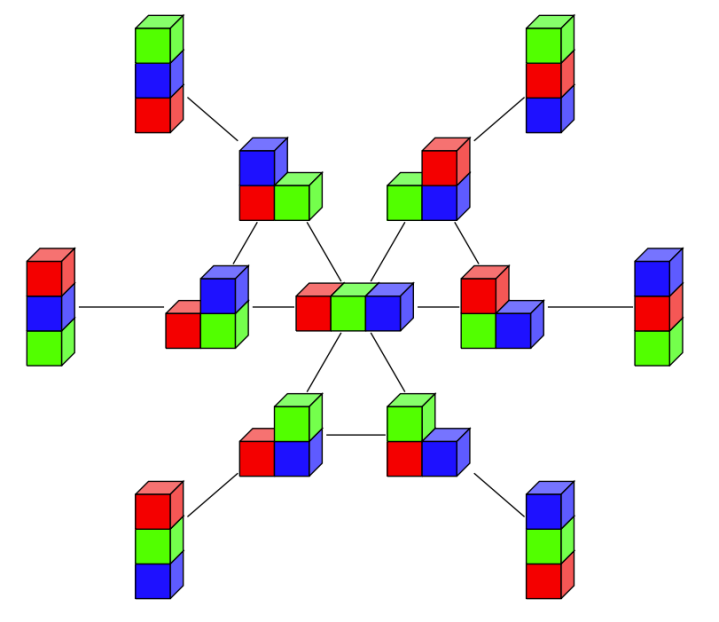
Why planning is difficult?

Solutions to deterministic planning problems are paths from an initial state to a goal state in the implicit **transition** graph, i.e., the graph that represents states and action-induced transitions between them.

Dijkstra's algorithm solves this problem in $O(|V| \log(|V|) + |E|)$

Are we done?

Not exactly $\Rightarrow$ $|V|$ of our interest is 10^{10}, 10^{20}, 10^{100}, ...



What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Why planning is difficult?

blocks	states
1	1
2	3
3	13
4	73
5	501
6	4051
7	37633
8	394353
9	4596553
...	
19	13564373693588558173

1. Finding a solution is polynomial time in the number of blocks (move everything onto the table and then construct the goal configuration).
2. Finding a shortest solution is NP-hard ...

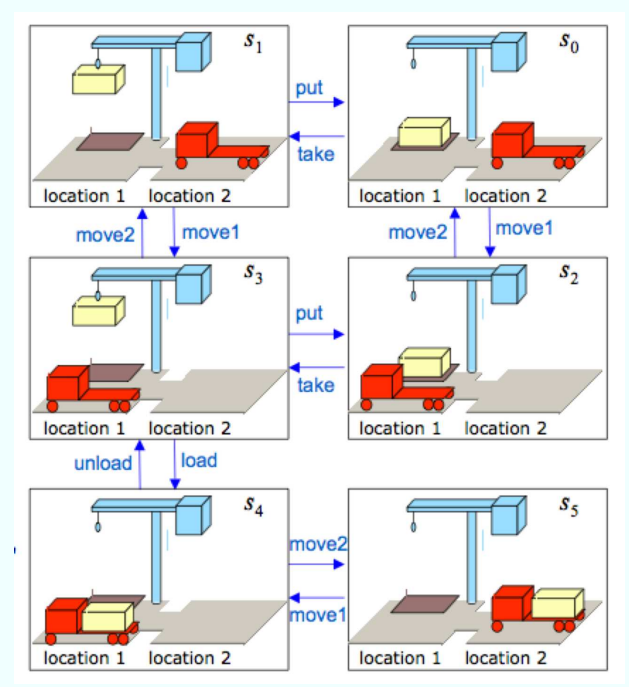
What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Why planning is difficult?

- Five locations, three robot carts, 100 containers, three piles
- $|V| \approx 10^{277}$

The number of atoms in the universe is only about 10^{87}

- The state space in our example is more than 10^{109} times as large !



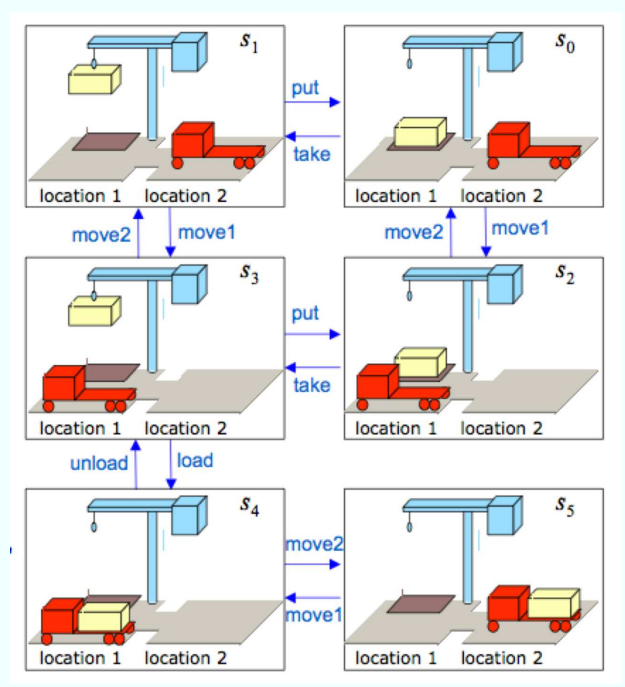
Why planning is difficult?

- Five locations, three robot carts, 100 containers, three piles
- $|V| \approx 10^{277}$

The number of atoms in the universe is only about 10^{87}

- The state space in our example is more than 10^{109} times as large !

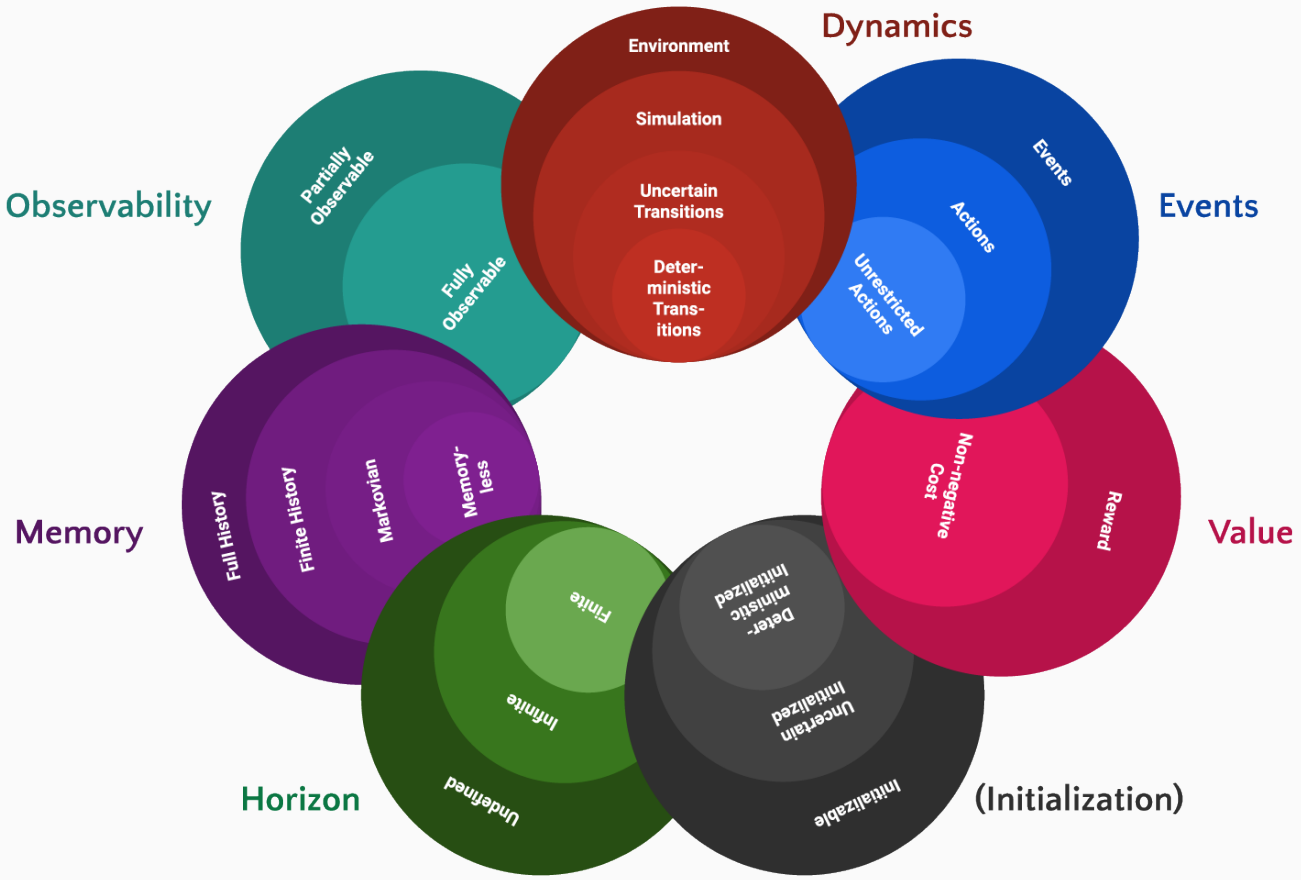
Very much not hopeless!



What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Approaches

Many Forms of Planning



https://github.com/fteicht/icaps24-skdecide-tutorial/tree/main

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Planning and sequential decision making

In the classical planning setting:

Plans (aka solutions) are sequences of moves that transform the initial state into the goal state

Intuitively, not all solutions are equally desirable

What is our task?

1. Find out whether there is a solution
2. Find any solution
3. Find an optimal solution
4. Find near-optimal solution
5. Fixed amount of time, find best solution possible
6. Find solution that satisfy property א
   (what is א? you choose!)

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Planning and sequential decision making

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

What is our task?

1. Find out whether there is a solution
2. Find any solution
3. Find an optimal solution
4. Find near-optimal solution
5. Fixed amount of time, find best solution possible
6. Find solution that satisfy property א
   (what is א? you choose!)

While all these tasks sound related, they are very different. The best-suited techniques are almost disjoint.

Classification of search algorithms

Uninformed search vs. heuristic search:

- **Uninformed search algorithms** only use the basic ingredients for general search algorithms
- **Heuristic search algorithms** additionally use **heuristic functions** which estimate how close a node is to the goal

Systematic search vs. local search:

- **Systematic algorithms** consider a large number of search nodes simultaneously
- **Local search algorithms** work with one (or a few) candidate solutions (search nodes) at a time
- not a black-and-white distinction; there are **crossbreeds** (e.g., enforced hill-climbing)

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

State Space / Graph Search

State-space search: one of the big success stories of AI

Must carefully distinguish two different problems:

- **Satisficing planning:** any solution is OK (although shorter solutions typically preferred)
- **Optimal planning:** plans must have shortest possible length

Both are often solved by search, but:

- details are **very different**
- almost **no overlap** between good techniques for satisficing planning and good techniques for optimal planning
- many problems that are trivial for satisficing planners are impossibly hard for optimal planners

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Required ingredients for search

A general search algorithm can be applied to any transition system for which we can define the following three operations:

- **init**$()$: generate the **initial state**
- **is-goal**(s): test if a given state is a **goal state**
- **succ**(s): generate the set of **successor states** of state s, along with the **actions** through which they are reached

Together, these three functions form a **search space** (a very similar notion to a transition system).

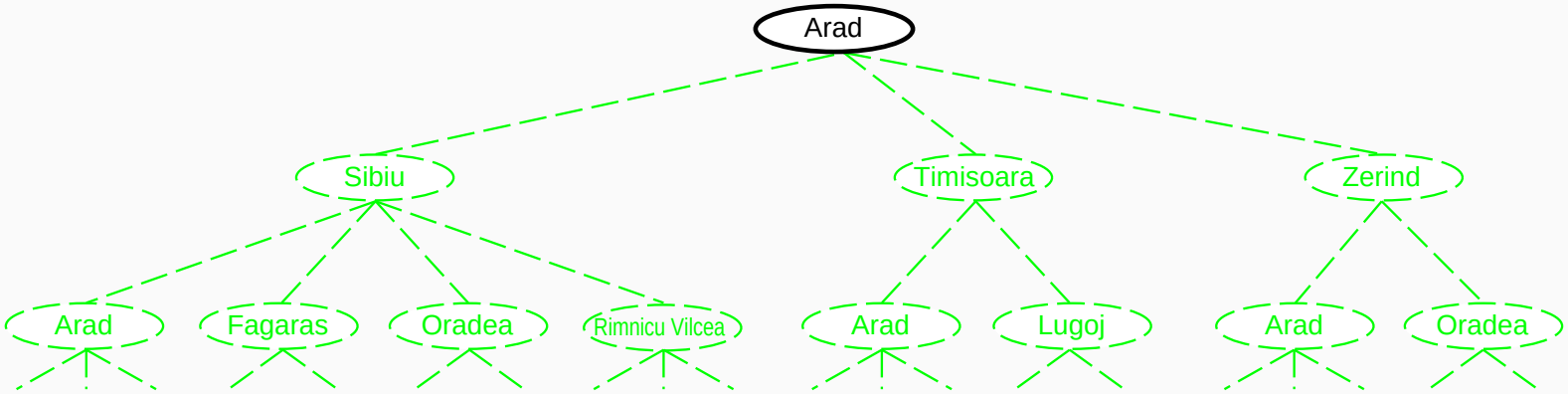
What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Planning by forward search: progression

Progression: Computing the successor state $f(s, a)$ of a state s with respect to an action a.

Progression planners find solutions by forward search:

- start from initial state
- iteratively pick a previously generated state and **progress it** through an action, generating a new state
- solution found when a goal state generated



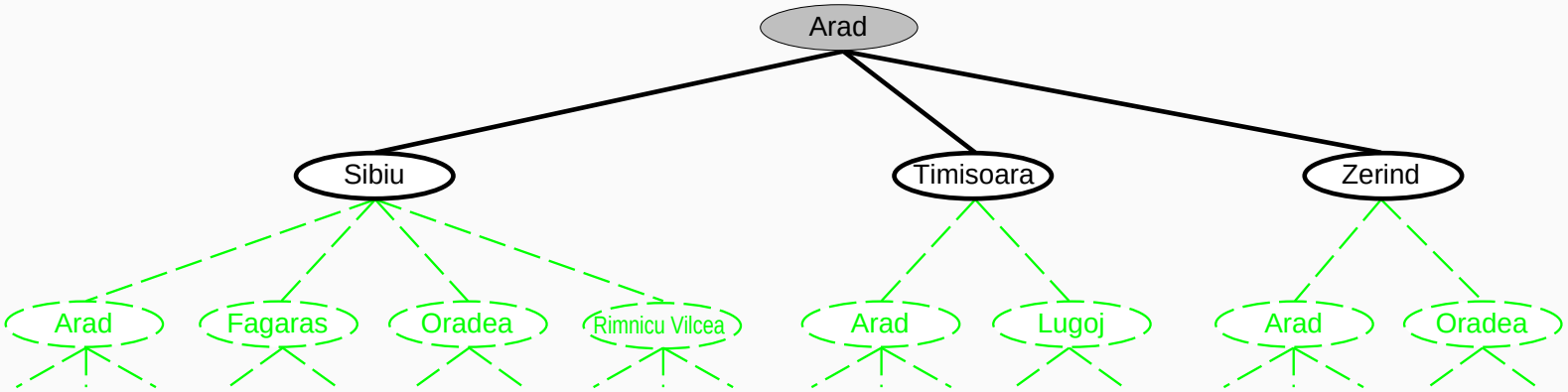
What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Planning by forward search: progression

Progression: Computing the successor state $f(s, a)$ of a state s with respect to an action a.

Progression planners find solutions by forward search:

- start from initial state
- iteratively pick a previously generated state and **progress it** through an action, generating a new state
- solution found when a goal state generated



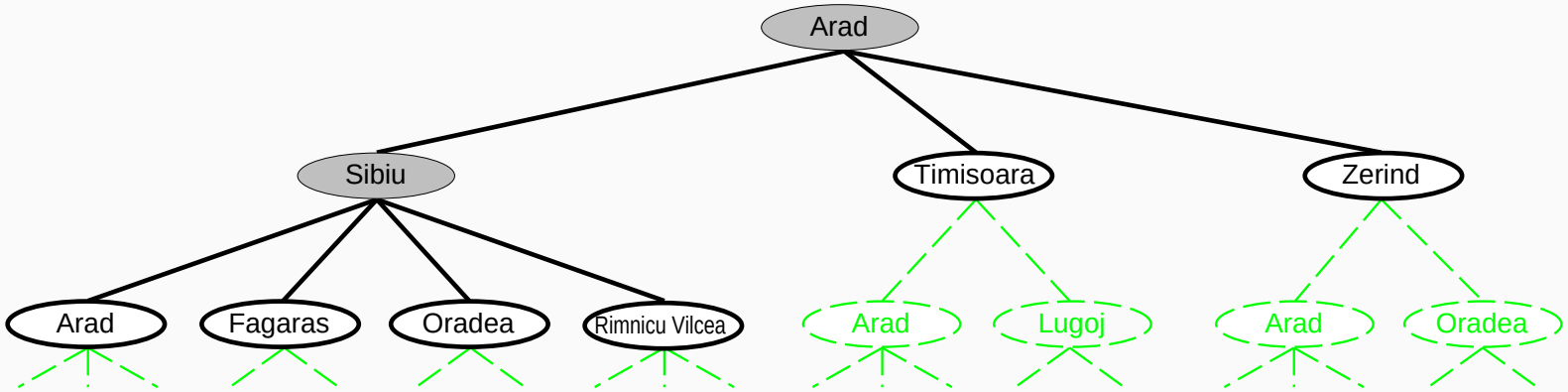
What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Planning by forward search: progression

Progression: Computing the successor state $f(s, a)$ of a state s with respect to an action a.

Progression planners find solutions by forward search:

- start from initial state
- iteratively pick a previously generated state and **progress it** through an action, generating a new state
- solution found when a goal state generated



What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Implementation: search states vs. search nodes

In search, one distinguishes:

- **search states** $s \rightsquigarrow$ states (vertices) of the transition system
- **search nodes** $\sigma \rightsquigarrow$ search states plus information on where/when/how they are encountered during search

What is in a search node?

Different search algorithms store different information in a search node σ, but typical information includes:

- $state(\sigma)$: associated search state
- $parent(\sigma)$: pointer to search node from which σ is reached
- $action(\sigma)$: an action leading from $state(parent(\sigma))$ to $state(\sigma)$
- $g(\sigma)$: cost (or length) of σ from the root node to σ)

For the root node, $parent(\sigma)$ and $action(\sigma)$ are undefined.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Common procedures for search algorithms

Before we describe the different search algorithms, we introduce three procedures used by all of them:

- **make-root-node:** Create a search node without parent.
- **make-node:** Create a search node for a state generated as the successor of another state.
- **extract-solution:** Extract a solution from a search node representing a goal state.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Procedure make-root-node

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

make-root-node: Create a search node without parent.

Procedure make-root-node

def make-root-node(s):

$\sigma :=$ **new** node

$state(\sigma) := s$

$parent(\sigma) :=$ undefined

$action(\sigma) :=$ undefined

$g(\sigma) := 0$

return σ

Procedure make-node

make-node: Create a search node for a state generated as the successor of another state.

Procedure make-node

def make-node(σ, o, s):

$\sigma' :=$ **new** node

$state(\sigma') := s$

$parent(\sigma') := \sigma$

$action(\sigma') := o$

$g(\sigma') := g(\sigma) + 1$

return σ'

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Procedure extract-solution

extract-solution: Extract a solution from a search node representing a goal state.

Procedure extract-solution

```
def extract-solution(σ):
    solution := new list
    while parent(σ) is defined:
        solution.push-front(action(σ))
        σ := parent(σ)
    return solution
```

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Tree Search

Procedure tree-search

```
def tree-search(problem, strategy):
    initialize the search tree using the initial state of problem
    loop:
        if there are no candidates for expansion
        then return failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state
            then return the corresponding solution
        else expand the node
            and add the resulting nodes to the search tree
```

Different implementations of $strategy$ lead to different algorithms.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Uninformed Search

Uninformed search algorithms

Popular uninformed systematic search algorithms:

- breadth-first search
- depth-first search
- iterated depth-first search

Popular uninformed local search algorithms:

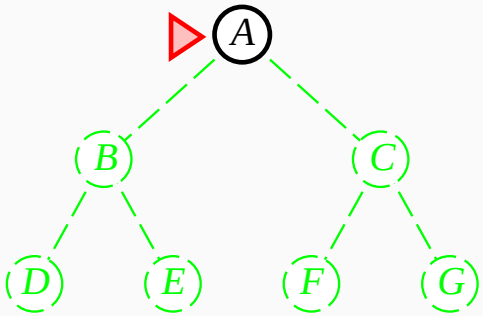
- random walk \hill-climbing.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Breadth-first search

Breadth-first search

queue := **new** fifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle a, s \rangle \in$ succ(*state*(σ)):
        $\sigma' :=$ make-node(σ, a, s)
        queue.push-back(σ')
return unsolvable

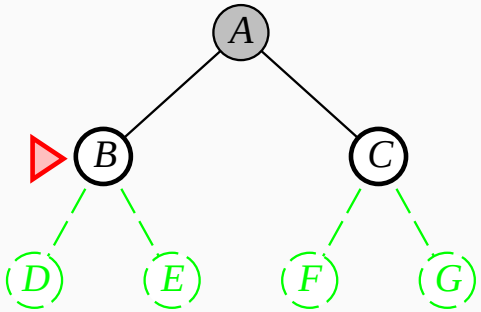


What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Breadth-first search

Breadth-first search

queue := **new** fifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle a, s \rangle \in$ succ(*state*(σ)):
        $\sigma' :=$ make-node(σ, a, s)
        queue.push-back(σ')
return unsolvable

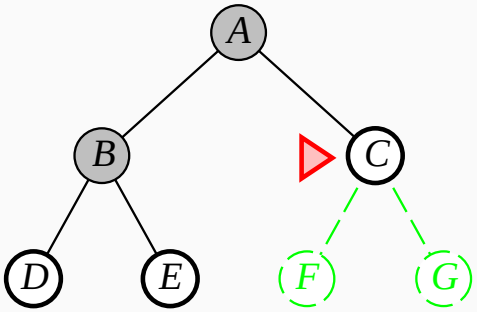


What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Breadth-first search

Breadth-first search

queue := **new** fifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle a, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, a, s)
        queue.push-back(σ')
return unsolvable

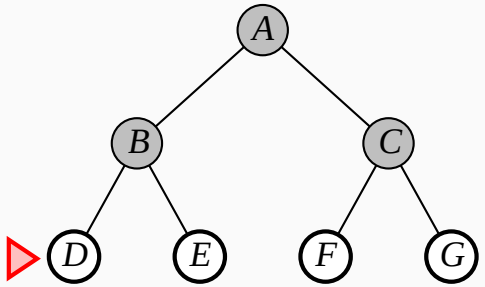


What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Breadth-first search

Breadth-first search

queue := **new** fifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle a, s \rangle \in$ succ(*state*(σ)):
        $\sigma' :=$ make-node(σ, a, s)
        queue.push-back(σ')
return unsolvable



What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Performance

Dimensions for evaluation

- **completeness**: always find a solution if one exists?
- **time complexity**: number of nodes generated/expanded
- **space complexity**: number of nodes in memory
- **optimality**: does it always find a least-cost solution?

Time/space complexity measured in terms of

b maximum branching factor of the search tree

d depth of the least-cost solution

m maximum depth of the state space (may be ∞)

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Properties of breadth-first search

Complete	Yes (if b is finite)
Time	$1 + b + b^2 + b^3 + \ldots + b^d + b(b^d - 1) = O(b^{d+1})$, i.e., $\exp(d)$
Space	$O(b^{d+1})$ *(why?)*
Optimal	Yes (if cost = 1 per step); can be generalized *(how?)*

Space is the big problem; can easily generate nodes at 100MB/sec so 24hrs = 8640GB.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Breadth-first search

Possible improvement: duplicate detection

Breadth-first search with duplicate detection

```
queue := new fifo-queue
queue.push-back(make-root-node(init()))
closed := ∅
while not queue.empty():
    σ = queue.pop-front()
    if state(σ) ∉ closed:
        closed := closed ∪ {state(σ)}
        if is-goal(state(σ)):
            return extract-solution(σ)
        for each ⟨a, s⟩ ∈ succ(state(σ)):
            σ′ := make-node(σ, a, s)
            queue.push-back(σ′)
return unsolvable
```

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Duplicate detection: Worth the effort?

Worst-case complexity

Time $1 + b + b^2 + b^3 + \ldots + b^d + b(b^d - 1) = O(b^{d+1})$

Same! ☺ *(why?)*

Space $O(b^{d+1})$

Same! ☹ *(why?)*

What is Model-based Planning

Planning Challenges

Approaches

Uninformed Search

Informed Search

Local search

Evaluating Planning Algorithms

Duplicate detection: Worth the effort?

Worst-case complexity

Time $1 + b + b^2 + b^3 + \ldots + b^d + b(b^d - 1) = O(b^{d+1})$

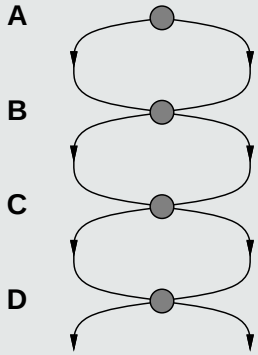
Same! ☺ *(why?)*

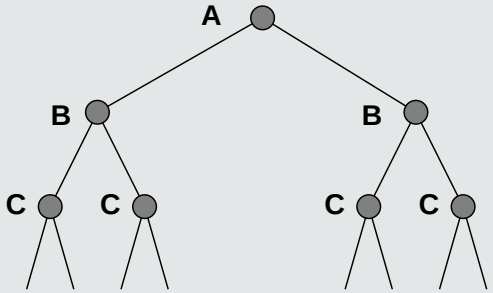
Space $O(b^{d+1})$

Same! ☹ *(why?)*

Empirical reality

Failure to detect repeated states can turn a linear problem into an exponential one!



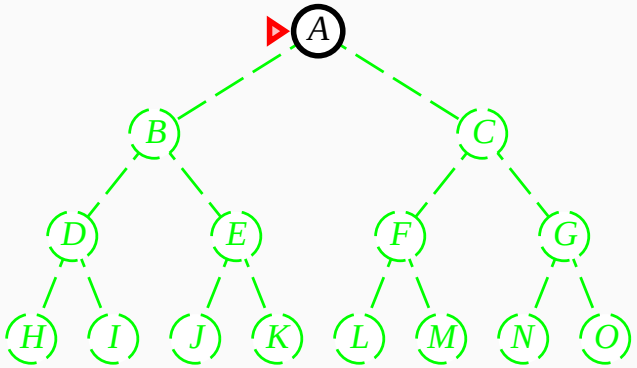


What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Depth-first search

Depth-first search

```
queue := new lifo-queue
queue.push-back(make-root-node(init()))
while not queue.empty():
    σ = queue.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each ⟨a, s⟩ ∈ succ(state(σ)):
        σ′ := make-node(σ, a, s)
        queue.push-front(σ′)
return unsolvable
```

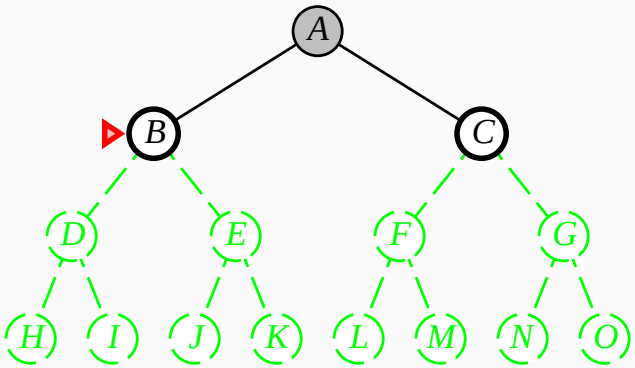


What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Depth-first search

Depth-first search

```
queue := new lifo-queue
queue.push-back(make-root-node(init()))
while not queue.empty():
    σ = queue.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each ⟨a, s⟩ ∈ succ(state(σ)):
        σ′ := make-node(σ, a, s)
        queue.push-front(σ′)
return unsolvable
```

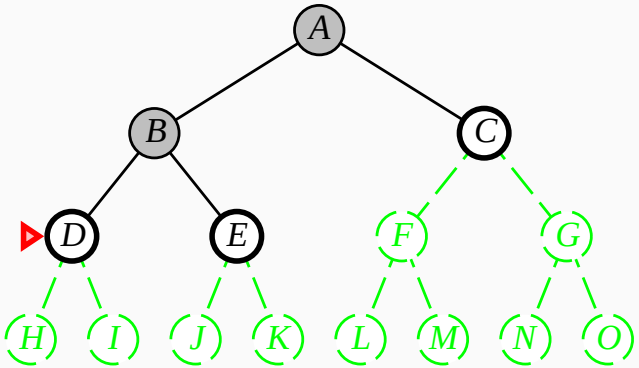


What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Depth-first search

Depth-first search

```
queue := new lifo-queue
queue.push-back(make-root-node(init()))
while not queue.empty():
    σ = queue.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each ⟨a, s⟩ ∈ succ(state(σ)):
        σ′ := make-node(σ, a, s)
        queue.push-front(σ′)
return unsolvable
```

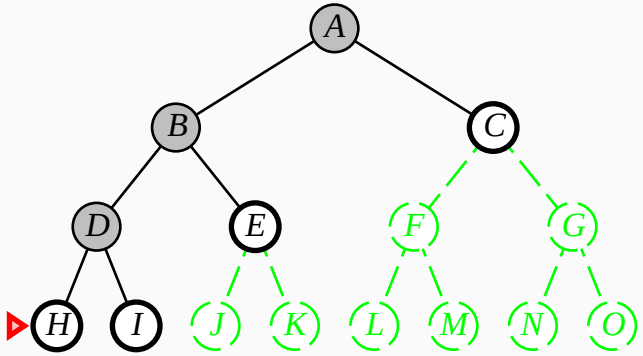


What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Depth-first search

Depth-first search

```
queue := new lifo-queue
queue.push-back(make-root-node(init()))
while not queue.empty():
    σ = queue.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each ⟨a, s⟩ ∈ succ(state(σ)):
        σ′ := make-node(σ, a, s)
        queue.push-front(σ′)
return unsolvable
```

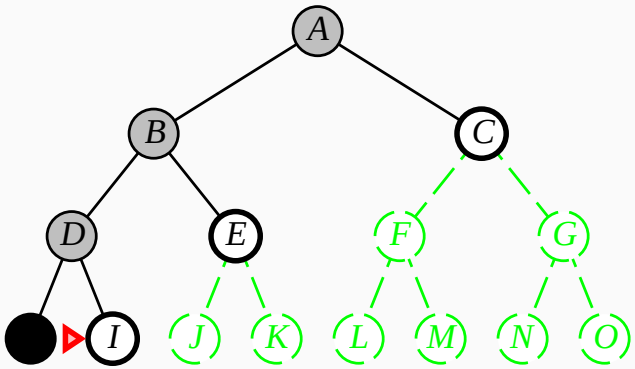


What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Depth-first search

Depth-first search

```
queue := new lifo-queue
queue.push-back(make-root-node(init()))
while not queue.empty():
    σ = queue.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each ⟨a, s⟩ ∈ succ(state(σ)):
        σ′ := make-node(σ, a, s)
        queue.push-front(σ′)
return unsolvable
```

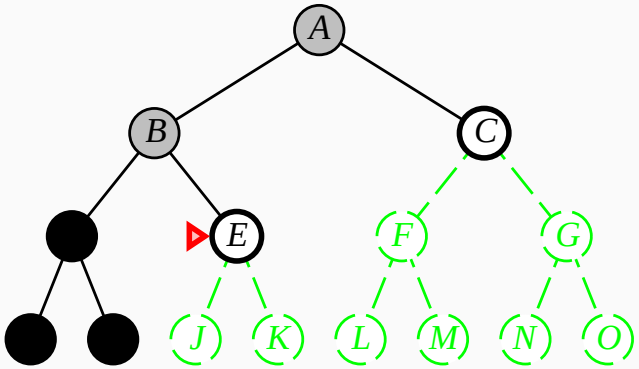


What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Depth-first search

Depth-first search

queue := new lifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle a, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, a, s)
        queue.push-front(σ')
return unsolvable

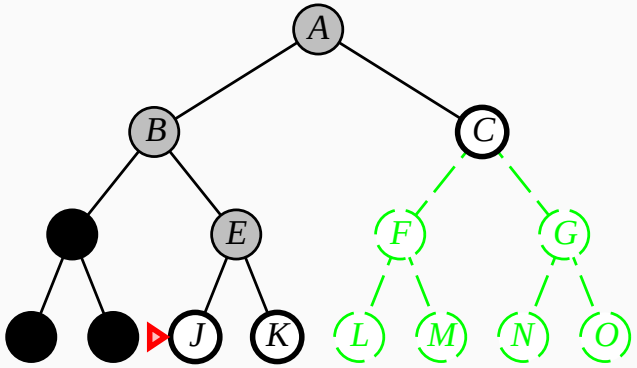


What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Depth-first search

Depth-first search

```
queue := new lifo-queue
queue.push-back(make-root-node(init()))
while not queue.empty():
    σ = queue.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each ⟨a, s⟩ ∈ succ(state(σ)):
        σ′ := make-node(σ, a, s)
        queue.push-front(σ′)
return unsolvable
```

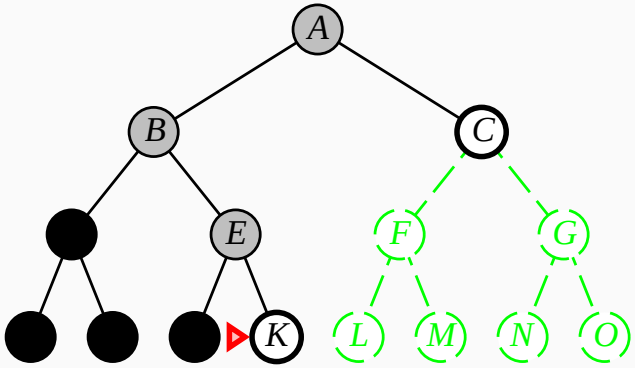


What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Depth-first search

Depth-first search

queue := new lifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle a, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, a, s)
        queue.push-front(σ')
return unsolvable

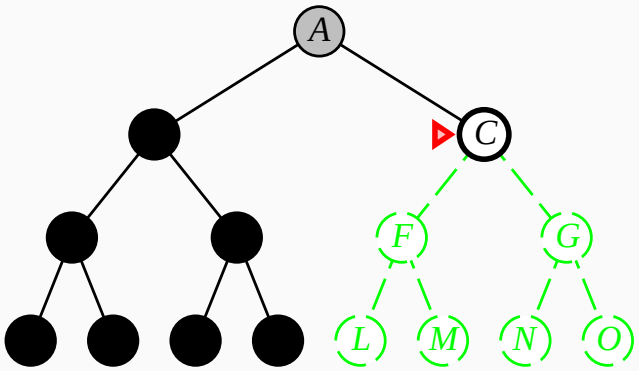


What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Depth-first search

Depth-first search

queue := new lifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle a, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, a, s)
        queue.push-front(σ')
return unsolvable

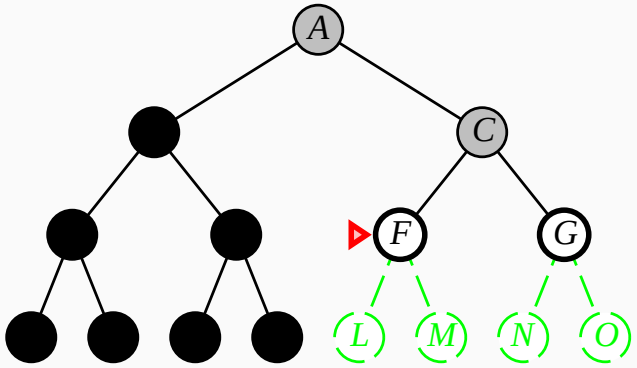


What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Depth-first search

Depth-first search

```
queue := new lifo-queue
queue.push-back(make-root-node(init()))
while not queue.empty():
    σ = queue.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each ⟨a, s⟩ ∈ succ(state(σ)):
        σ′ := make-node(σ, a, s)
        queue.push-front(σ′)
return unsolvable
```

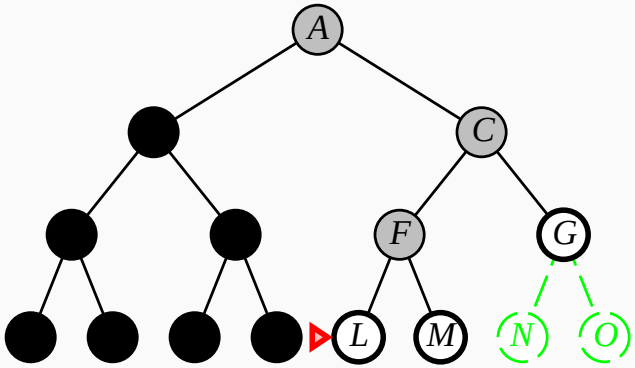


What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Depth-first search

Depth-first search

```
queue := new lifo-queue
queue.push-back(make-root-node(init()))
while not queue.empty():
    σ = queue.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each ⟨a, s⟩ ∈ succ(state(σ)):
        σ′ := make-node(σ, a, s)
        queue.push-front(σ′)
return unsolvable
```

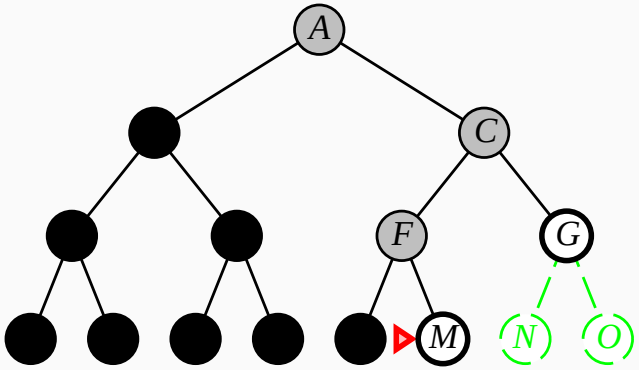


What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Depth-first search

Depth-first search

```
queue := new lifo-queue
queue.push-back(make-root-node(init()))
while not queue.empty():
    σ = queue.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each ⟨a, s⟩ ∈ succ(state(σ)):
        σ′ := make-node(σ, a, s)
        queue.push-front(σ′)
return unsolvable
```



What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Properties of depth-first search

Complete No: fails in spaces with $b = \infty$ or $m = \infty$, spaces with loops

- Possible improvement: duplicate detection along path $\rightsquigarrow$ complete in finite spaces

Time $O(b^m)$

- terrible if m is much larger than d, but
- if solutions are dense, may be much faster than BFS

Space $O(bm) \rightsquigarrow$ **linear space!**

Optimal No *(right?)*

Informed Search

Informed Search

So far, we have seen graph-search algorithms that identify optimal plans by **blindly exploring** the state space.

Informed search methods use **heuristic estimates**, extracted automatically from the problem description, to guide the search toward promising states.

Heuristics with formal guarantees can induce guarantees on the search algorithm.

Nevertheless, even heuristics without strong formal guarantees can still be highly useful in practice.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Informed Search

Well-known systematic heuristic search algorithms:

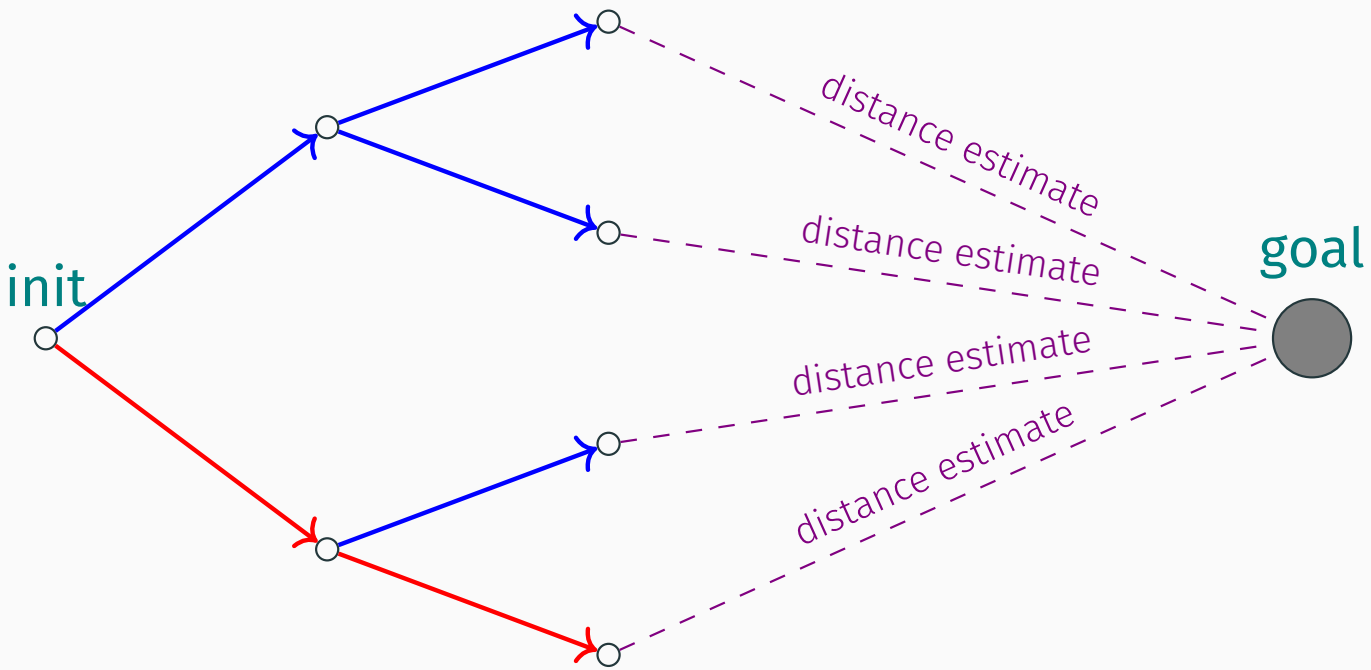
- greedy best-first search
- A^*
- weighted A^*
- $ID - A^*$
- depth-first branch-and-bound search
- breadth-first heuristic search

Well-known heuristic local search algorithms:

- hill-climbing
- enforced hill-climbing
- beam search
- tabu search
- genetic algorithms
- simulated annealing

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Heuristic search: intuition



What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Required ingredients for heuristic search

A **heuristic search algorithm** requires one more operation in addition to the definition of a search space.

Definition (heuristic function)

Let Σ be the set of nodes of a given search space.

A **heuristic function** or **heuristic** (for that search space) is a function $h : \Sigma \to \mathbb{N}_0 \cup \{\infty\}$.

In shortest-path problems, $h(\sigma)$ estimates the **remaining distance to a goal** from node σ.

Typically, heuristics are computed based only on the underlying state $h(\sigma) \stackrel{\text{def}}{=} h(state(\sigma))$

Heuristics guide search toward
promising regions of the state space.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

What exactly is a heuristic estimate?

What does it mean that h "estimates the distance to goal"?

- For most heuristic search algorithms, h does not need to have any strong properties for the algorithm to work.
- However, the **efficiency** of the algorithm closely relates to how accurately h reflects the actual goal distance.
- For some algorithms, like A^*, we can prove strong formal relationships between properties of h and properties of the algorithm (optimality, dominance, run-time for bounded error, ...)
- For other search algorithms, "it works well in practice" is often as good an analysis as one gets.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Perfect heuristic

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Let Σ be the set of nodes of a given search space.

Definition (optimal/perfect heuristic)

The **optimal** or **perfect heuristic** of a search space is the heuristic h^* which maps each search node σ to the length of a shortest path from *state*(σ) to any goal state.

Note: $h^*(\sigma) = \infty$ iff no goal state is reachable from σ.

Properties of heuristics

A heuristic h is called

- **safe** if for all $\sigma \in \Sigma$ $h(\sigma) = \infty$ implies $h^*(\sigma) = \infty$
- **goal-aware** if $h(\sigma) = 0$ for all goal nodes $\sigma \in \Sigma$
- **admissible** if $h(\sigma) \leq h^*(\sigma)$ for all nodes $\sigma \in \Sigma$
- **consistent** if $h(\sigma) \leq h(\sigma') + cost(\sigma, \sigma')$
  for all nodes $\sigma, \sigma' \in \Sigma$ such that σ' is a successor of σ

Relationships?

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Heuristic search algorithms

In the following, we use the following for shortest-path algorithms.

- $g(\sigma)$ Distance from start.
- $h(\sigma)$ Heuristic function that estimates the expected distance to the nearest goal.

Our focus is on shortest-path problems, so we focus on estimating the cost to a goal state.
In settings in which the goal is to maximize some objective, $h(\sigma)$ may be used to estimate the value that can be acquired from that node.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Greedy best-first search

Chooses the next node to expand based on $f(n) = h(n)$.

Greedy best-first search (with duplicate detection)

```
open := new min-heap ordered by (σ ↦ h(σ))
open.insert(make-root-node(init()))
closed := ∅
while not open.empty():
    σ = open.pop-min()
    if state(σ) ∉ closed:
        closed := closed ∪ {state(σ)}
        if is-goal(state(σ)):
            return extract-solution(σ)
        for each ⟨a, s⟩ ∈ succ(state(σ)):
            σ′ := make-node(σ, a, s)
            if h(σ′) < ∞:
                open.insert(σ′)
return unsolvable
```

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Properties of GBFS

- Complete ?
- Optimal ?

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Properties of GBFS

- **Complete ?** for safe heuristics (due to duplicate detection)
- **Optimal ? Suboptimal** unless h satisfies some very strong assumptions (similar to being perfect)
- **Invariant** under all strictly monotonic transformations of h (e.g., scaling with a positive constant or adding a constant)

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

A*

Chooses the next node to expand based on $f(\sigma) = g(\sigma) + h(\sigma)$

Key points:

- If the heuristic is admissible, then A^* is guaranteed to return an optimal solution.
- Trade-off between estimation quality and computation cost.
- Example to popular heuristic in motion planning: straight-line / Manhattan-distance.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

A^* (with duplicate detection and reopening)

```
open := new min-heap ordered by (σ ↦ f(σ) = g(σ) + h(σ))
open.insert(make-root-node(init()))
closed := ∅
distance := ∅
while not open.empty():
    σ = open.pop-min()
    if state(σ) ∉ closed or g(σ) < distance(state(σ)):
        closed := closed ∪ {state(σ)}
        distance(state(σ)) := g(σ)
        if is-goal(state(σ)):
            return extract-solution(σ)
        for each ⟨a, s⟩ ∈ succ(state(σ)):
            σ′ := make-node(σ, a, s)
            if h(σ′) < ∞:
                open.insert(σ′)
return unsolvable
```

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

A^*

A version with one node maintained per state replaces the min-heap with a mean-set-heap.

The first line becomes:
open := **new** min-set-heap ordered by $(\sigma \mapsto f(\sigma) = g(\sigma) + h(\sigma))$

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Properties of A^*

The most commonly used algorithm for optimal planning.

Complete ?

Optimal?

Re-expansions?

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Properties of A^*

The most commonly used algorithm for optimal planning.

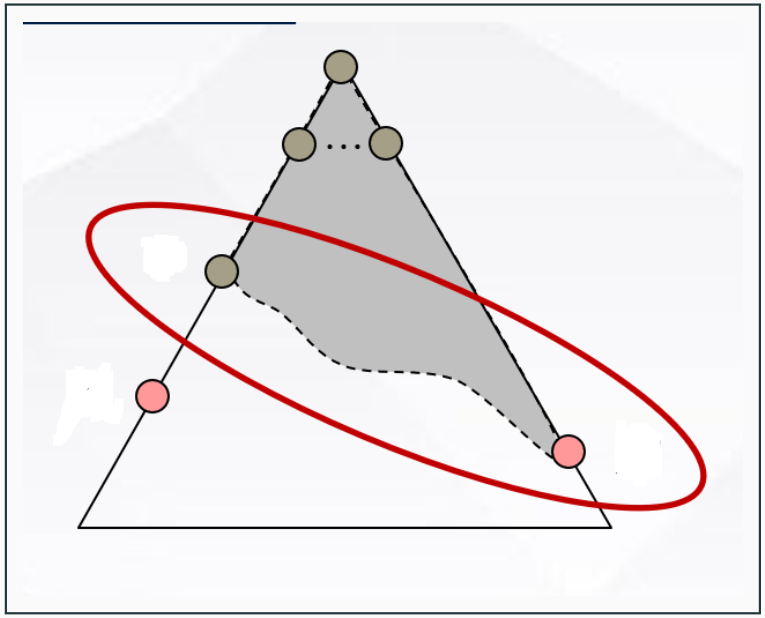
Complete ? for safe heuristics (even without duplicate detection).

Optimal? for admissible heuristics.

Re-expansions? never reopens nodes if h is consistent.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Optimality of Tree-Search A^* with admissible h



What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Optimality of Tree-Search A^* with admissible h

Suppose to the contrary that the algorithm returns a suboptimal plan via *extract-solution*(σ) for some node σ with *state*$(\sigma) \in G$.

- If so, then no optimal goal node σ^* was in the open list.
- If so, then open list contains some unexpanded node σ' on the (optimal) path from root node to σ^*.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Optimality of Tree-Search A^* with admissible h

Suppose to the contrary that the algorithm returns a suboptimal plan via *extract-solution*(σ) for some node σ with *state*$(\sigma) \in G$.

- If so, then no optimal goal node σ^* was in the open list.
- If so, then open list contains some unexpanded node σ' on the (optimal) path from root node to σ^*.

$$\begin{aligned} f(\sigma) = \quad & g(\sigma) && \text{since } h(\sigma) = 0 \\ > \quad & g(\sigma^*) && \text{since } \sigma \text{ is suboptimal} \\ \geq \quad & f(\sigma') && \text{since } h \text{ is admissible} \end{aligned}$$

$f(\sigma) > f(\sigma') \rightsquigarrow$ contradiction with "σ is dequeued before σ'"

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Weighted A^*

Weighted A^* (with duplicate detection and reopening)

```
open := new min-heap ordered by (σ ↦ g(σ) + W · h(σ))
open.insert(make-root-node(init()))
closed := ∅
distance := ∅
while not open.empty():
    σ = open.pop-min()
    if state(σ) ∉ closed or g(σ) < distance(state(σ)):
        closed := closed ∪ {state(σ)}
        distance(σ) := g(σ)
        if is-goal(state(σ)):
            return extract-solution(σ)
        for each ⟨a, s⟩ ∈ succ(state(σ)):
            σ′ := make-node(σ, a, s)
            if h(σ′) < ∞:
                open.insert(σ′)
return unsolvable
```

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Properties of weighted A^*

The **weight** $W \in \mathbb{R}_0^+$ is a parameter of the algorithm.

- for $W = 0$, behaves like breadth-first search
- for $W = 1$, behaves like A^*
- for $W \to \infty$, behaves like greedy best-first search

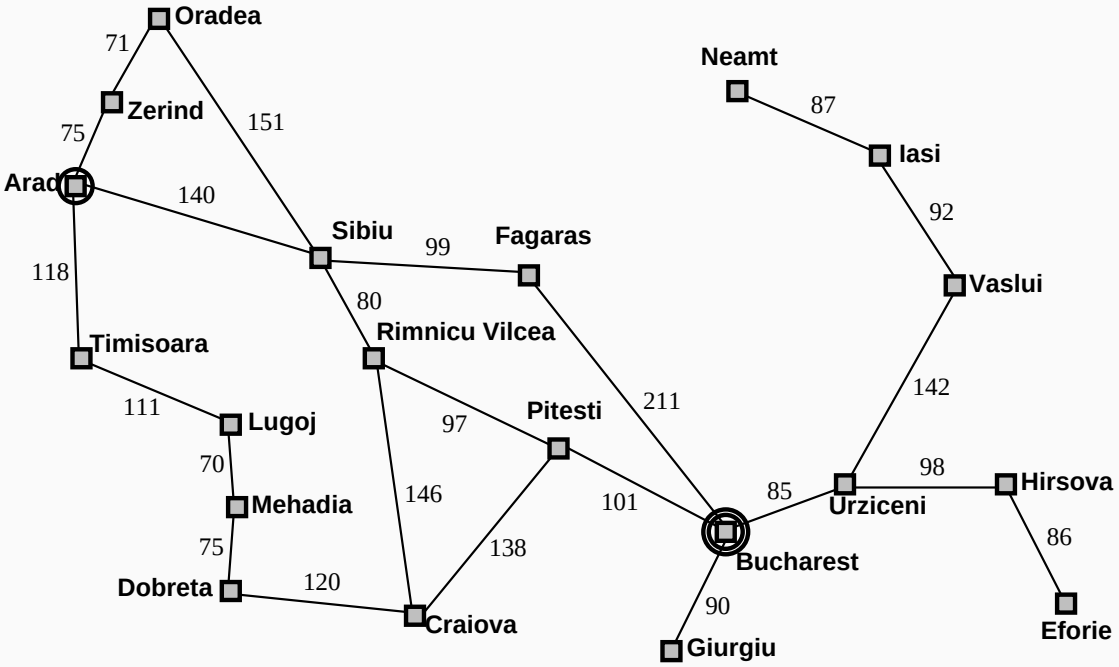
Properties:

- A common algorithm for satisficing planning.
- For $W > 1$, can prove similar properties to A^*, **bounded suboptimal**: generated solutions are at most a factor W as long as optimal ones.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Example

Planning a trip to Bucharest.



What is the solution generated by each approach?

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Best First Search

Best-First Search is a **general search framework** that underlies many of the algorithms we have seen so far.

It applies to both:

- **cost minimization** (e.g., shortest-path problems), and
- **utility maximization** under constraints (e.g., budget).

The key idea is to prioritize nodes using an **evaluation function** $f(\cdot)$.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Best-First Search - Minimizing Cost to Goal

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Best-First Search

```
open := new min-heap ordered by (σ ↦ f(σ))
open.insert(make-root-node(init()))
while not open.empty():
    σ = open.pop-min()
        if is-goal(state(σ)):
            return extract-solution(σ)
        for each ⟨a, s⟩ ∈ succ(state(σ)):
            σ′ := make-node(σ, a, s)
            open.insert(σ′)
return unsolvable
```

Closed list can be added.

Best-First Search - Maximizing Utility

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Best-First Search

```
open := new max-heap ordered by (σ ↦ f(σ))
open.insert(make-root-node(init()))
while not open.empty():
    σ = open.pop-max()
        if is-terminal(state(σ)):
            return extract-solution(σ)
        for each ⟨a, s⟩ ∈ succ(state(σ)):
            σ′ := make-node(σ, a, s)
            open.insert(σ′)
return failure
```

Best-First Search: Instantiations

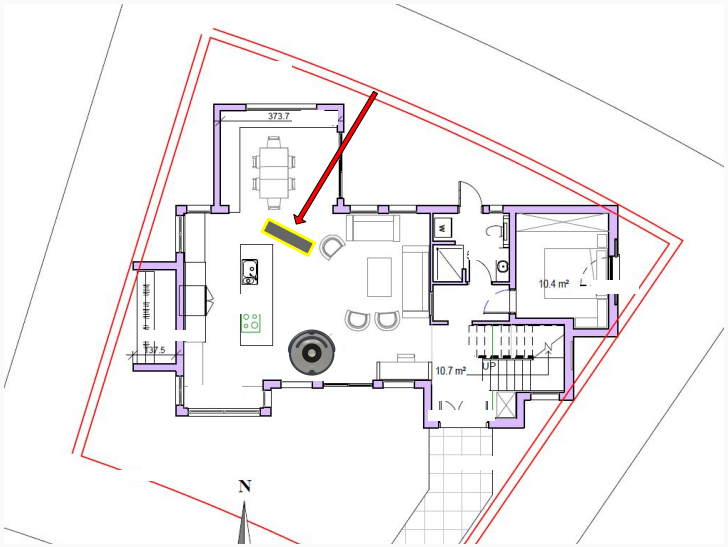
- **Uniform-Cost Search:** $f(\sigma) = g(\sigma)$
- **Greedy Best-First Search:** $f(\sigma) = h(\sigma)$
- A^*: $f(\sigma) = g(\sigma) + h(\sigma)$
- **Weighted A^*:** $f(\sigma) = g(\sigma) + W \cdot h(\sigma)$

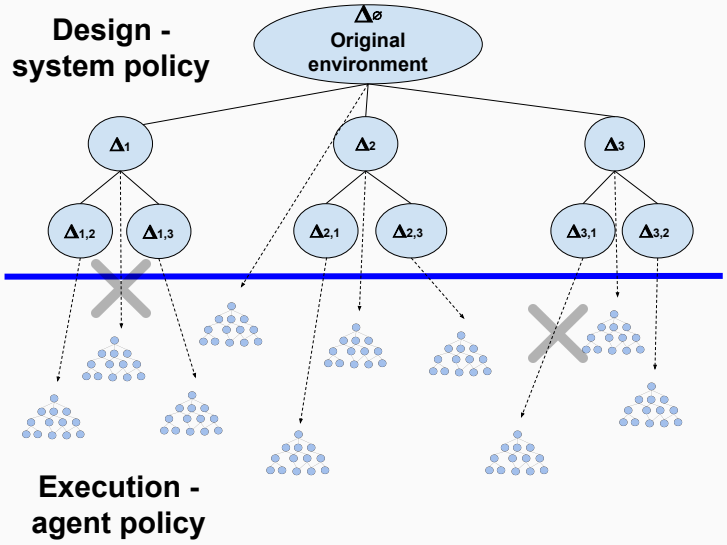
In utility-maximizing settings, $f(\sigma)$ represents value rather than cost.

Different algorithms = different evaluation functions $f(\sigma)$

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Best First Design (Keren et al.)





Find a path that represents a sequence of environment modifications that **maximizes** a user-specified utility measure.

Admissible heuristics **over-estimate** the value of a modification sequence.

We use $f = h$ (greedy prioritization), since feasibility is constrained by a design budget.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Best First Design (Keren et al.)

Best-First Design (BFD)

open := **new** max-heap ordered by $(\sigma \mapsto h(\sigma))$
$\sigma_{cur} := \langle design, \vec{m}_{\emptyset} \rangle$
while σ_{cur}:
    if IsExecution(σ_{cur}):
        return $\sigma_{cur}.\vec{m}$
    for each $\sigma_{suc} \in$ GetSuccessors(σ_{cur}, δ):
        open.insert($\langle design, \sigma_{suc}.\vec{m} \rangle, h(\sigma_{suc})$)
    if $\Phi_{\sigma}(\sigma_{cur}.\vec{m}) = 1$: [path is valid]
        open.insert($\langle execution, \vec{m}_{new} \rangle, v^*(\delta_{\vec{m}_{new}})$)
    σ_{cur} := *open*.extract-max()
return error

design is a flag indicating whether the node represents a design or execution node.

If *IsExecution* is true, the node is terminal.

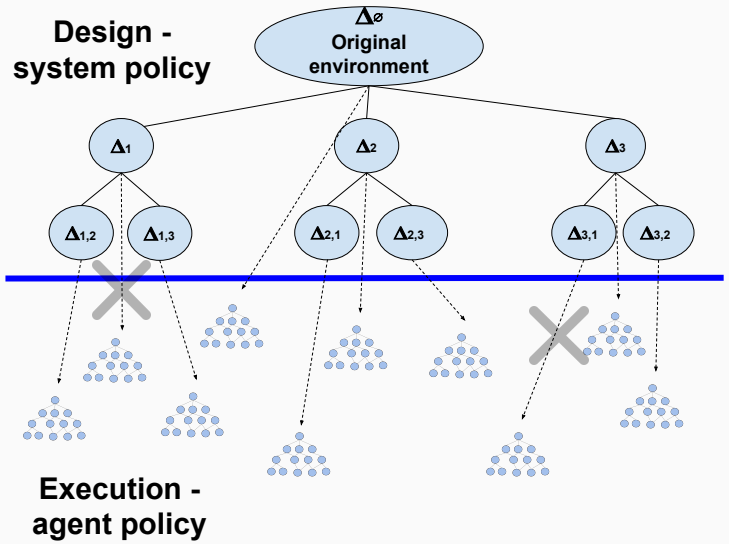
Φ is an indicator that specifies the allowed modification

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Best First Design (Keren et al.)

For optimality, we require h to be admissible.

An admissible estimation of a design node σ never underestimates the maximal system's utility in the environment it represents (the original environment to which the sequence of modifications represented by σ is applied).



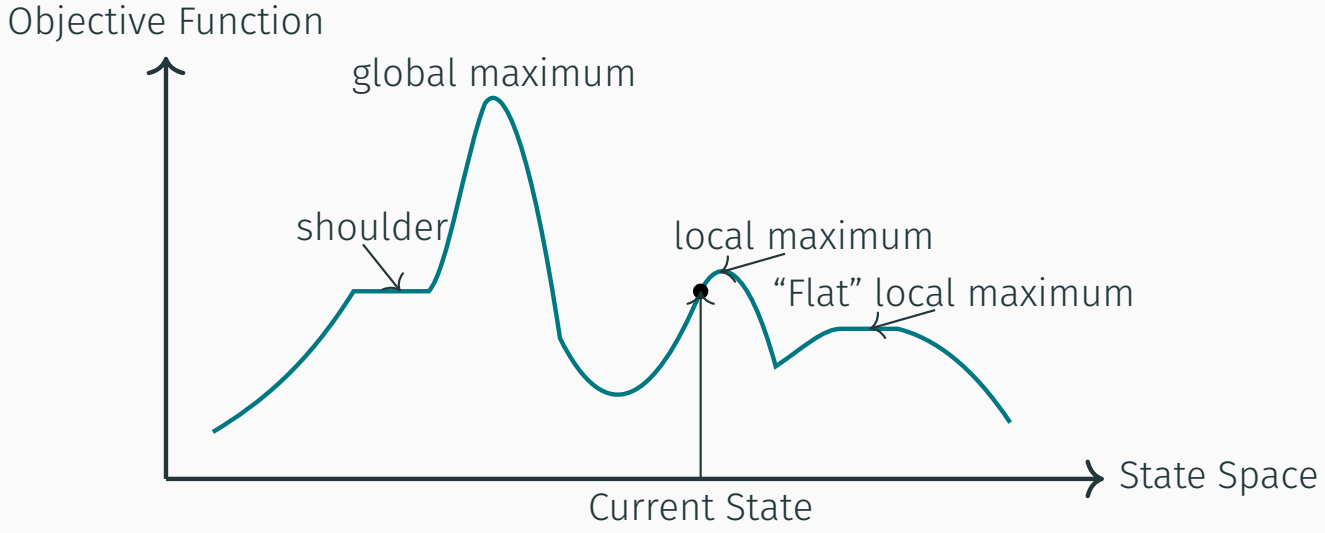
In maximization settings, admissibility means **over-estimating** achievable utility (optimistic upper bound).

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Local search

Hill-climbing

A local search algorithm that incrementally improves the solution by exploring the neighboring states of the current state.



What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Hill-climbing

Hill-climbing

$\sigma :=$ make-root-node(init())
forever:
  if is-goal(state(σ)):
    return extract-solution(σ)
  $\Sigma' := \{$ make-node$(\sigma, a, s) \mid \langle a, s\rangle \in$ succ(state(σ)) $\}$
  $\sigma :=$ an element of Σ' minimizing h (random tie breaking)

- Can get stuck in **local minima** where immediate improvements of $h(\sigma)$ are not possible
- Many variations: tie-breaking strategies, restarts

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Enforced hill-climbing

Enforced hill-climbing: procedure improve

```
def improve(σ_0):
    queue := new fifo-queue
    queue.push-back(σ_0)
    closed := ∅
    while not queue.empty():
        σ = queue.pop-front()
        if state(σ) ∉ closed:
            closed := closed ∪ {state(σ)}
            if h(σ) < h(σ_0):
                return σ
            for each ⟨a, s⟩ ∈ succ(state(σ)):
                σ' := make-node(σ, a, s)
                queue.push-back(σ')
    fail
```

$\rightsquigarrow$ breadth-first search for more promising node than σ_0

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Enforced hill-climbing (ctd.)

Enforced hill-climbing

$\sigma :=$ make-root-node(**init**())
while not **is-goal**(state(σ)):
    $\sigma :=$ improve(σ)
return extract-solution(σ)

- A common algorithm for satisficing planning.
- Can fail if procedure *improve* fails (when the goal is unreachable from σ_0)
- Complete for **undirected** search spaces (where the successor relation is symmetric) if $h(\sigma) = 0$ for all goal nodes and only for goal nodes

Classification: what works where for (deterministic) planning?

Uninformed vs. heuristic search:

Systematic search vs. local search:

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Classification: what works where for (deterministic) planning?

Uninformed vs. heuristic search:

- For **satisficing** planning, heuristic search vastly outperforms uninformed algorithms on most domains.
- For **optimal** planning, heuristic search typically outperforms uninformed algorithms, but an efficiently implemented uninformed algorithm is not easy to beat in most domains.

Systematic search vs. local search:

- For **satisficing** planning, the most successful algorithms are somewhere between the two extremes.
- For **optimal** planning, systematic algorithms are required.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Where heuristics come from?

General idea

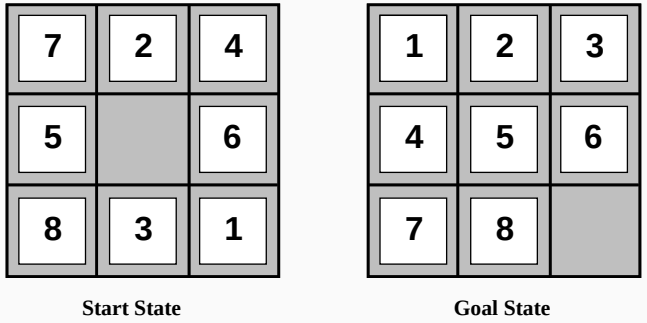
(Admissible) heuristic functions obtained as
(optimal) cost functions of **relaxed problems**

Examples

- Euclidian distance in Path Finding
- Manhattan distance in N-puzzle
- Spanning Tree in Traveling Salesman Problem
- Shortest Path in Job Shop Scheduling

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

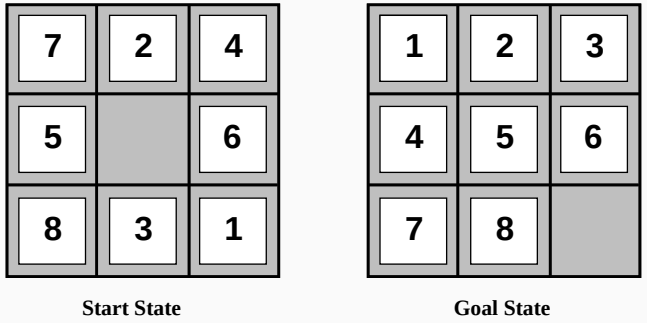
Example



- A tile can move from square A to square B if A is adjacent to B and B is blank $\rightsquigarrow$ solution distance h^*
- A tile can move from square A to square B if A is adjacent to B $\rightsquigarrow$ manhattan distance heuristic h^{MD}
- A tile can move from square A to square B $\rightsquigarrow$ misplaced tiles heuristic h^{MT}

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Example



- A tile can move from square A to square B if A is adjacent to B and B is blank $\leadsto$ solution distance h^*
- A tile can move from square A to square B if A is adjacent to B $\leadsto$ manhattan distance heuristic h^{MD}
- A tile can move from square A to square B $\leadsto$ misplaced tiles heuristic h^{MT}

Here: $h^*(s_0) = ?$, $h^{MD}(s_0) = 14$, $h^{MT}(s_0) = 6$
In general, $h^* \geq h^{MD} \geq h^{MT}$. *(Why?)*

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Relaxations as Heuristics

Relaxations can be used for heuristic estimations.

Different possibilities:

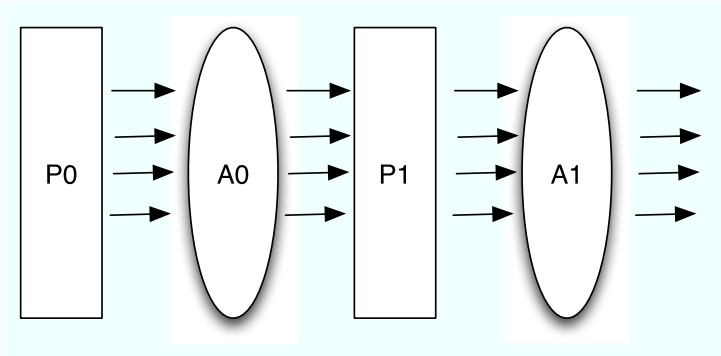
- Implement an **optimal planner** for relaxed planning tasks and use its solution lengths as an estimate, even though it is NP-hard.
  $\rightsquigarrow$ h^+ **heuristic** -ingore delete effects.
- Do not actually solve the relaxed planning task, but compute an estimate of its difficulty in a different way.
  $\rightsquigarrow$ h_{max} **heuristic**, h_{add} **heuristic**
- Compute a solution for relaxed planning tasks which is not necessarily optimal, but "reasonable".
  $\rightsquigarrow$ h_{FF} **heuristic**

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Graphical "interpretation": Relaxed planning graphs

- Build a layered **reachability graph** $P_0, A_0, P_1, A_1, \ldots$

$$
\begin{aligned}
P_0 &= \{p \in I\} \\
A_i &= \{a \in A \mid pre(a) \subseteq P_i\} \\
P_{i+1} &= P_i \cup \{p \in add(a) \mid a \in A_i\}
\end{aligned}
$$



- Terminate when $G \subseteq P_i$

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Running example

$$P = \{a, b, c, d, e, f, g, h\}$$
$$I = \{a\}$$
$$G = \{c, d, e, f, g\}$$
$$a_1 = \langle \{a\}, \{b, c\}, \{a\} \rangle$$
$$a_2 = \langle \{a, c\}, \{d\}, \{d\} \rangle$$
$$a_3 = \langle \{b, c\}, \{e\}, \{e, f\} \rangle$$
$$a_4 = \langle \{b\}, \{f\}, \emptyset \rangle$$
$$a_5 = \langle \{d\}, \{e, f\}, \{d\} \rangle$$
$$a_6 = \langle \{d\}, \{g\}, \{b\} \rangle$$

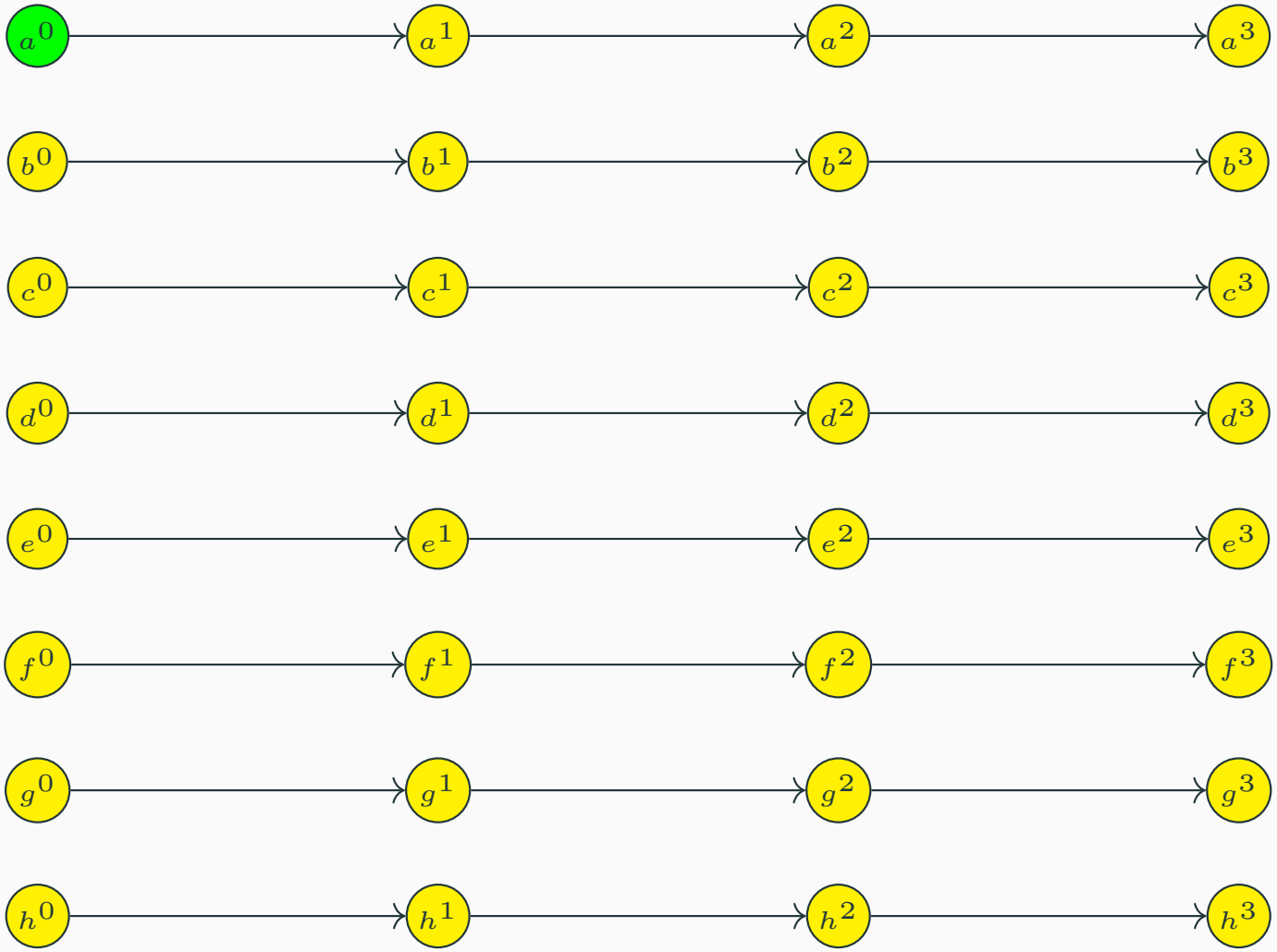
What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Running example

$$P = \{a, b, c, d, e, f, g, h\}$$
$$I = \{a\}$$
$$G = \{c, d, e, f, g\}$$
$$a_1 = \langle \{a\}, \{b, c\}, \emptyset \rangle$$
$$a_2 = \langle \{a, c\}, \{d\}, \emptyset \rangle$$
$$a_3 = \langle \{b, c\}, \{e\}, \emptyset \rangle$$
$$a_4 = \langle \{b\}, \{f\}, \emptyset \rangle$$
$$a_5 = \langle \{d\}, \{e, f\}, \emptyset \rangle$$
$$a_6 = \langle \{d\}, \{g\}, \emptyset \rangle$$

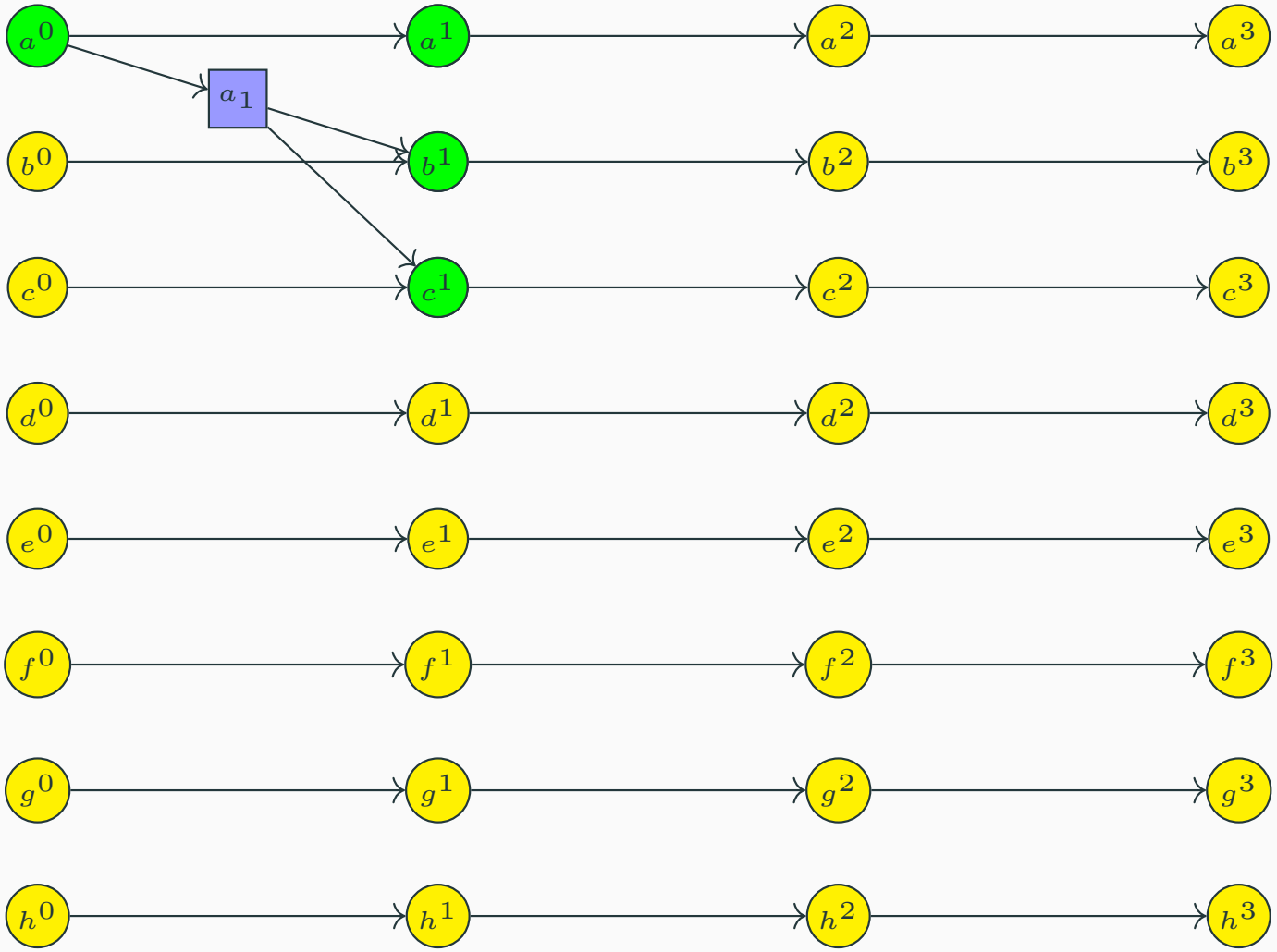
What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Running example: Relaxed planning graph



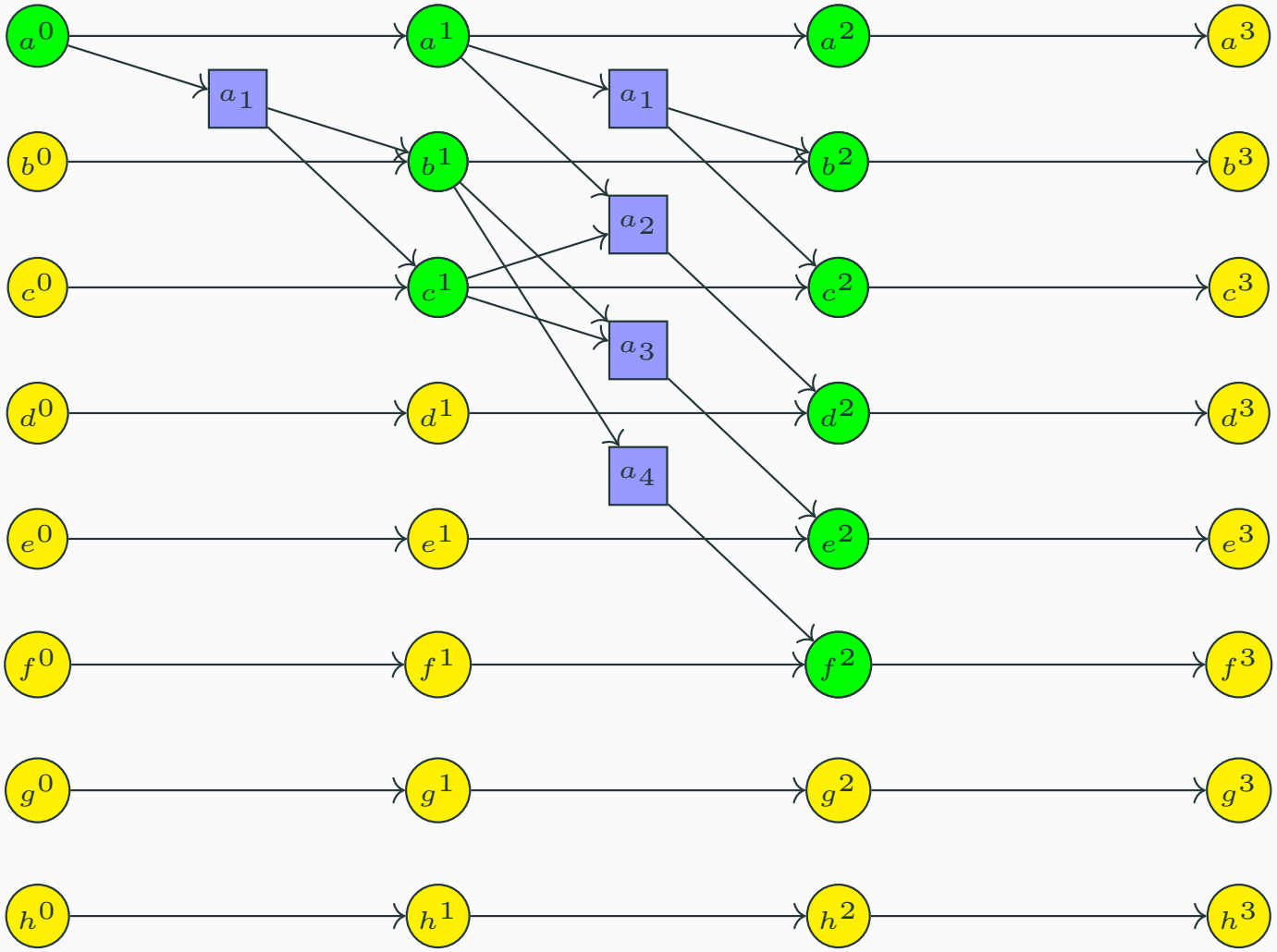
What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Running example: Relaxed planning graph



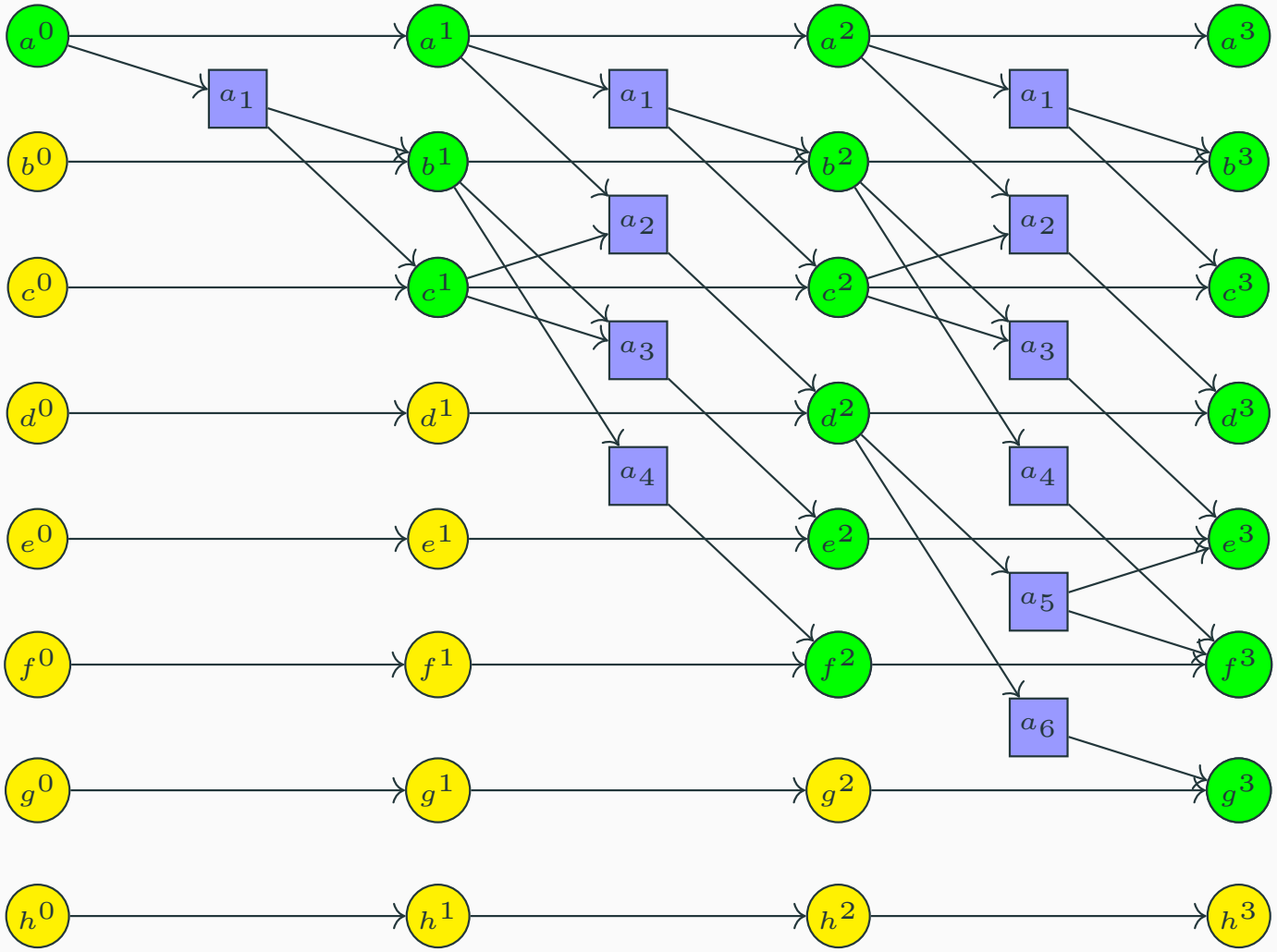
What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Running example: Relaxed planning graph



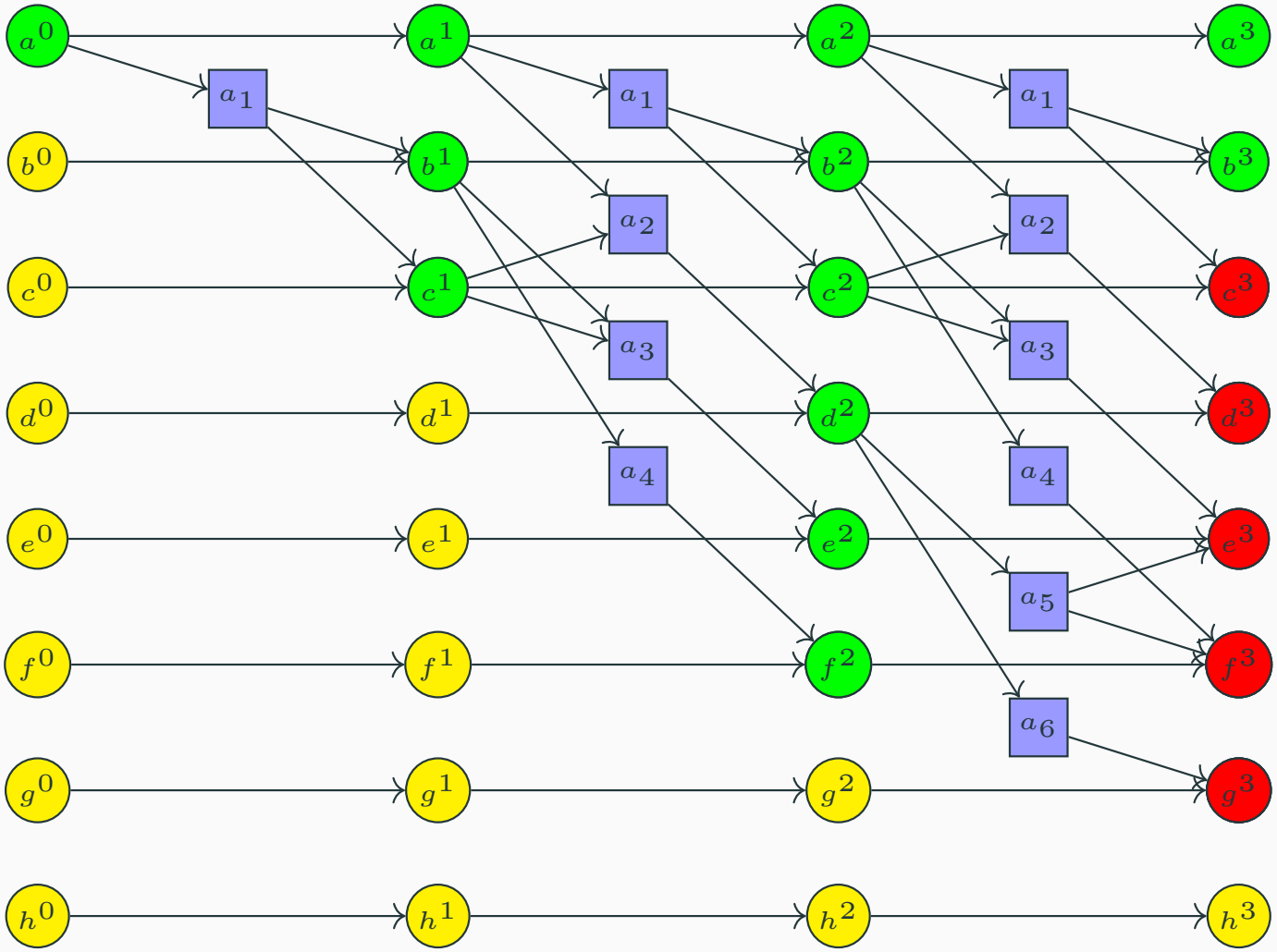
What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Running example: Relaxed planning graph



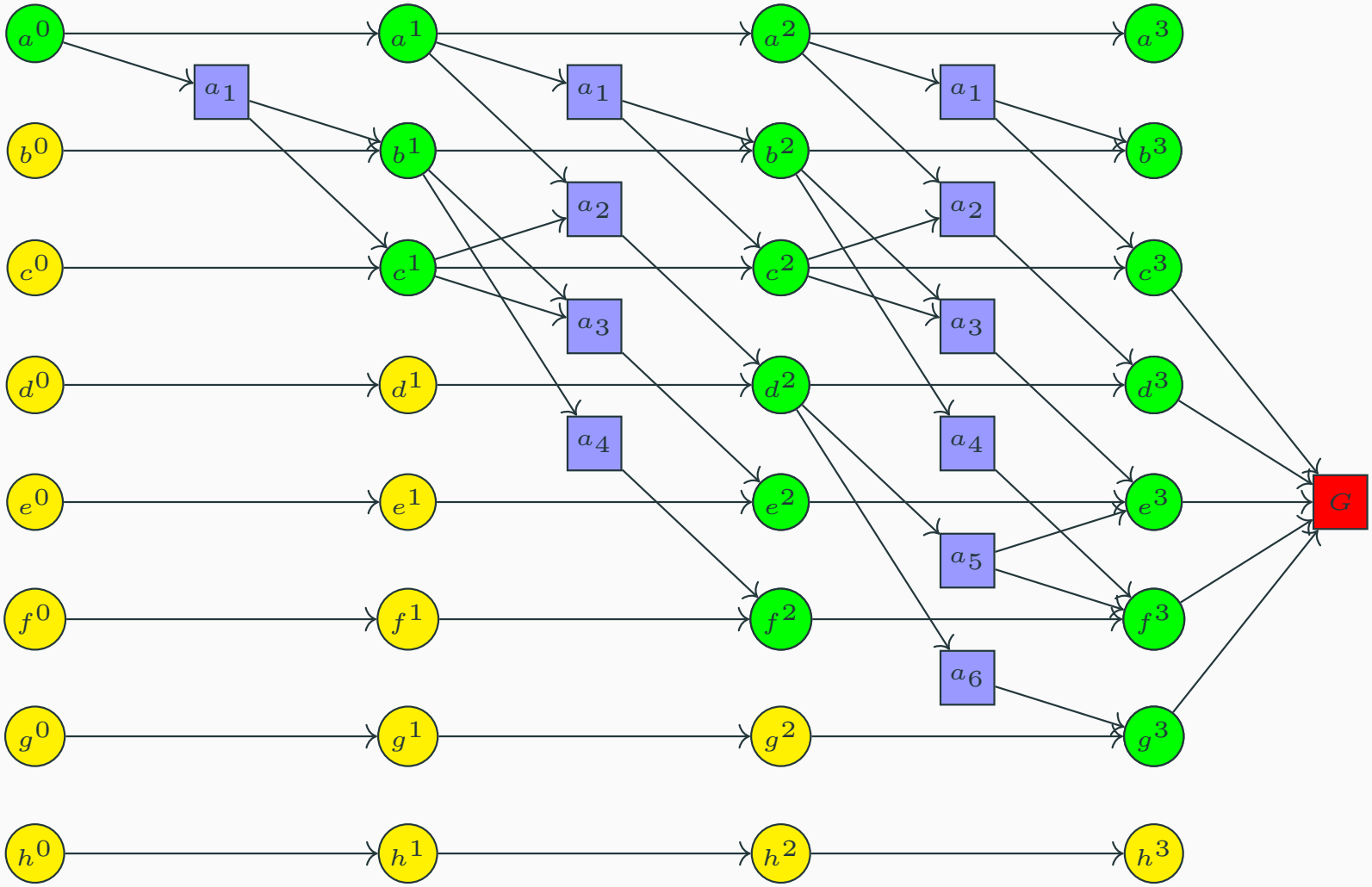
What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Running example: Relaxed planning graph



What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Running example: Relaxed planning graph



What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Dominance relation between admissible heuristics

Precision matters

Given two admissible heuristics h_1, h_2, if $h_2(\sigma) \geq h_1(\sigma)$ for all search nodes σ, then h_2 **dominates** h_1 and is better for optimizing search

Typical search costs (unit-cost action)

$h^*(I) = 14$	BFS $\approx$ 1,700,000 nodes
	A*$(h_1) \approx$ 560 nodes
	A*$(h_2) \approx$ 115 nodes
$h^*(I) = 24$	BFS $\approx$ 27,000,000,000 nodes
	A*$(h_1) \approx$ 40,000 nodes
	A*$(h_2) \approx$ 1,650 nodes

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Dominance relation between admissible heuristics

Precision matters

Given two admissible heuristics h_1, h_2, if $h_2(\sigma) \geq h_1(\sigma)$ for all search nodes σ, then h_2 **dominates** h_1 and is better for optimizing search

Combining admissible heuristics

For any admissible heuristics $h_1, \ldots, h_k$,

$$h(\sigma) = \max_{i=1}^{k}\{h_i(\sigma)\}$$

is also admissible and dominates all individual h_i

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

Formulating Heuristics

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

General idea

(Admissible) heuristic functions obtained as (optimal) cost functions of relaxed problems

- OK, but heuristic is **yet another input** to our agent!
- Satisfactory for general solvers?
- Satisfactory in special-purpose solvers?

Formulating Heuristics

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Heuristics
Evaluating Planning Algorithms

General idea

(Admissible) heuristic functions obtained as (optimal) cost functions of relaxed problems

- OK, but heuristic is **yet another input** to our agent!
- Satisfactory for general solvers?
- Satisfactory in special-purpose solvers?

Towards domain-independent agents

- How to get heuristics **automatically**?
- Can such automatically derived heuristics **dominate** the domain-specific heuristics crafted by hand?

Formulating Heuristics

What is Model-based Planning

Planning Challenges

Approaches

Uninformed Search

Informed Search

Local search

Heuristics

Evaluating Planning Algorithms

Evaluating Planning Algorithms

Evaluating Planning Algorithms

Dimensions for evaluation

- **Completeness**: always find a solution if one exists?
- **Time complexity**: number of nodes generated/expanded
- **Space complexity**: number of nodes in memory
- **Optimality**: does it always find a least-cost solution?
- **Anytime**: does the solution improve the more resources are used ?

Time/space complexity measured in terms of

b maximum branching factor of the search tree

d depth of the least-cost solution

m maximum depth of the state space (may be ∞)

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Evaluating Algorithms: Modern Considerations

Robustness: how sensitive is performance to noise, uncertainty, or modeling errors?

Scalability: how does performance change as the problem size grows?

Adaptivity: can the algorithm improve or adjust based on experience?

Generality: does the method apply broadly across domains or rely on strong assumptions?

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Evaluating Algorithms: Modern Considerations

No exploration vs. exploitation since the model is assumed to be known.

No account for uncertainty or risk since the model includes deterministic actions.

Interpretability is inherent, since the model is given.

No explicit account for energy efficiency beyond standard analysis of worst-case complexity.

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms

Conclusion

Planning can be formulated as **search in a transition graph** that is not provided explicitly.

Best-First Search provides a unifying framework for many planning algorithms via an evaluation function $f(\cdot)$

Different choices of f and h lead to different behaviors:

- cost minimization (e.g., shortest-path problems)
- utility maximization under constraints

Heuristics are critical for **scalability**, but introduce trade-offs between optimality and efficiency

What is Model-based Planning
Planning Challenges
Approaches
Uninformed Search
Informed Search
Local search
Evaluating Planning Algorithms